

LIVA: Hệ thống được hỗ trợ LLM đa tác nhân để phân tích lỗ hổng IoT

Zhe Yang,¹ Hao Peng,² Thành viên cấp cao, IEEE, Yanling Jiang,³ Jianwei Liu,⁴ Thành viên cấp cao, IEEE, Hongbin Luo,⁵ Mingsheng Tang,⁶ Thành viên, IEEE, Jiahe Li và Kun Zhang⁷

Tóm tắt—Các thiết bị IoT đã được tích hợp sâu vào cuộc sống hàng ngày của chúng ta, khiến việc nghiên cứu bảo mật toàn diện về các thiết bị cơ sở hạ tầng quan trọng ngày càng trở nên quan trọng. Các kỹ thuật phân tích tĩnh, đặc biệt là những kỹ thuật tận dụng sự lan truyền vết bản, đã cho thấy nhiều hứa hẹn trong việc xác định các lỗ hổng bảo mật trong các thiết bị này, phát hiện một cách hiệu quả các lỗ hổng nghiêm trọng. Tuy nhiên, các giải pháp hiện tại thường gặp phải những hạn chế về cả hiệu quả phát hiện và độ chính xác. Để giải quyết những thách thức này, bài viết này giới thiệu Liva, một công cụ phân tích vết bản tĩnh mới được thiết kế để phát hiện các lỗ hổng web trong thiết bị IoT. Liva sử dụng phương pháp tiếp cận đa tác nhân mô hình ngôn ngữ lớn (LLM) để phân tích vết bản nhị phân tĩnh, chủ yếu tận dụng các mô hình nguồn mở được tinh chỉnh và LLM thương mại để cải thiện khả năng nhận dạng nguồn/bồn rửa và phân tích dữ liệu vết bản—các lĩnh vực mà các phương pháp truyền thống thường thiếu sót—do đó nâng cao hiệu quả phân tích tổng thể. Công cụ phân tích cốt lõi của LIVA tận dụng mô hình nguồn mở Qwen3-32B đã được tinh chỉnh bằng cách sử dụng bộ dữ liệu gồm 3.000 mẫu thiết bị trong thế giới thực. Mô hình được tinh chỉnh này đạt được độ chính xác được cải thiện 3 điểm phần trăm trong việc xác định các mối quan hệ truyền bá dữ liệu không rõ ràng so với LLM thương mại, đồng thời tăng hiệu quả phân tích trung bình lên 5,5%. Một đánh giá toàn diện về Liva, được thực hiện trên tập dữ liệu gồm 64 thiết bị từ 11 nhà cung cấp, cho thấy rằng nó đã phát hiện ra nhiều lỗ hổng được biết đến nhiều hơn lần lượt là 309 và 349 so với các giải pháp tiên tiến SaTC và Karonte, đồng thời giảm tỷ lệ dương tính giả lần lượt là 59,4% và 67,6%. Liva đạt được tỷ lệ thu hồi 98,1% và độ chính xác 74,6%, đồng thời $6.7 \times$ giảm thời gian phân tích so với đường cơ sở hoạt động tốt nhất. Hơn nữa, trong lĩnh vực phát hiện lỗ hổng zero-day, Liva đã phát hiện ra 64 lỗ hổng chưa được biết trước đó, 39 trong số đó đã được gán mã nhận dạng CVE/CNVD chính thức.

Điều khoản chỉ mục—Internet of Things, phân tích vết bản, mô hình ngôn ngữ lớn, lỗ hổng, tinh chỉnh.

I. GIỚI THIỆU

T Sự phát triển nhanh chóng của công nghệ Internet of Things (IoT) đang thay đổi cuộc sống con người. Số lượng thiết bị được kết nối toàn cầu đã vượt qua 10 tỷ và dự kiến sẽ tăng

đạt 27 tỷ người vào năm 2025 [1]. Việc mở rộng này đã dẫn đến một cơ sở hạ tầng mạng quan trọng bao gồm bộ định tuyến, điểm truy cập Wi-Fi, cổng VPN và tường lửa. Các thiết bị biên này chịu trách nhiệm định tuyến, mã hóa và bảo mật dữ liệu, được tiếp xúc trực tiếp với các mạng công cộng, khiến chúng trở thành mục tiêu hàng đầu cho các cuộc tấn công mạng. Sự phổ biến ngày càng tăng của các thiết bị này đặt ra những thách thức bảo mật đáng kể cho các hệ thống nhúng. “Báo cáo an ninh mạng năm 2025” của Check Point cho thấy các cuộc tấn công mạng toàn cầu đã tăng 44% kể từ năm 2023, trong đó 37% nhắm mục tiêu cụ thể vào các thiết bị biên và cơ sở hạ tầng mạng [2]. Các thiết bị biên, đặc biệt là bộ định tuyến, cổng VPN và thiết bị IoT, đã trở thành điểm truy cập quan trọng đối với những kẻ tấn công [3], nhấn mạnh bản chất thiết yếu của nghiên cứu bảo mật tập trung vào các thiết bị quan trọng của IoT.

Trong lĩnh vực phân tích bảo mật thiết bị IoT, phân tích vết bản đặc biệt quan trọng do ba đặc điểm cơ bản của hệ sinh thái phần mềm: bản chất nguồn đóng, sự phụ thuộc vốn có của phần cứng và mức độ phổ biến của các mẫu lỗ hổng cụ thể. Hầu hết các thiết bị IoT thương mại đều dựa vào phần sụn nguồn đóng, điều này cản trở việc mô phỏng động hiệu quả [4], [5]. Ví dụ: mô phỏng dịch vụ mạng của FirmAE [6] cho phần mềm điều khiển Linux chỉ đạt được tỷ lệ thành công 79% [6], do đó hạn chế khả năng áp dụng các kỹ thuật làm mờ động. Hơn nữa, các lỗ hổng phần mềm IoT thường biểu hiện hành vi giống như vết bản [7], [8], trong đó các đầu vào bên ngoài chưa được xác minh (ví dụ: các gói mạng) truyền dọc theo các đường dẫn phụ thuộc dữ liệu đến các điểm vận hành nhạy cảm như chức năng thao tác bộ nhớ hoặc giao diện thực thi lệnh [9]. Trong bối cảnh này, phân tích vết bản nhị phân tĩnh mang lại những ưu điểm đáng chú ý: bằng cách phân tích trực tiếp luồng lệnh của phần sụn, nó thiết lập các mối quan hệ luồng dữ liệu chức năng chéo mà không yêu cầu mã nguồn hoặc mô phỏng phần cứng, cho phép truy tìm các đường dẫn hoàn chỉnh từ các nguồn vết bản (ví dụ: `webGetVar`) đến các phần chứa nhạy cảm (ví dụ: `strcpy`) [10], [11]. Một số công cụ tiên tiến, bao gồm KARONTE [12], SaTC [13] và Emtaint [14], đã chứng minh tính thực tiễn cao hơn các giải pháp động trong việc phân tích bảo mật phần mềm IoT. Tuy nhiên, các phương pháp phân tích vết bản tĩnh hiện tại vẫn gặp phải một số hạn chế cản trở tính hiệu quả của chúng trong các tình huống IoT trong thế giới thực.

Việc xác định điểm nguồn và điểm đích là một thách thức, thường gây ra kết quả âm tính giả. Sự đa dạng của nguồn đầu vào trong các thiết bị IoT và tính chất không thể đoán trước của cơ chế định tuyến của chúng làm phức tạp việc xác định chính xác các điểm nguồn [15]. Hơn nữa, việc đóng gói các điểm chìm nguy hiểm tiềm tàng trong các thư viện dùng chung cản trở việc nhận dạng và xử lý tiếp theo của chúng [16], [17].

Nhận ngày 9 tháng 9 năm 2025; sửa đổi ngày 16 tháng 12 năm 2025; chấp nhận ngày 12 tháng 2 năm 2026. Ngày xuất bản 16 tháng 2 năm 2026; ngày của phiên bản hiện tại là 12 tháng 5 năm 2026. Công trình này được hỗ trợ bởi Quỹ khoa học tự nhiên Bắc Kinh dưới sự tài trợ L253021, một phần bởi NSFC theo tài trợ 62322202 và tài trợ U25B2029, một phần bởi Quỹ phát triển khoa học và công nghệ địa phương của tỉnh Hà Bắc do Chính phủ trung ương hướng dẫn của Trung Quốc theo tài trợ 254Z9902G và một phần bởi Dự án kế hoạch khoa học và công nghệ Thạch Gia Trang thuộc tài trợ 2511301807A. (Tác giả tương ứng: Hao Peng.)

Các tác giả làm việc tại Trường Khoa học và Công nghệ Mạng, Đại học Beihang, Bắc Kinh 100191, Trung Quốc (e-mail: zheyang.yz@foxmail.com; peng_hao@buaa.edu.cn; yanlingj@buaa.edu.cn; liujianwei@buaa.edu.cn; luohb@buaa.edu.cn; tangmingsheng@buaa.edu.cn; hi_jacklee@buaa.edu.cn; zhangkun 9605@foxmail.com).

Bài viết này có tài liệu bổ sung có thể tải xuống tại <https://doi.org/10.1109/TDSC.2026.3665343> do tác giả cung cấp. Bộ nhận dạng đối tượng kỹ thuật số 10.1109/TDSC.2026.3665343

Phân tích bảo mật của các thiết bị IoT có hiệu quả thấp trong việc phân tích lan truyền dữ liệu bị nhiễm độc. Các mối quan hệ gọi hàm phức tạp thường dẫn đến các tính toán dư thừa đáng kể khi phân tích độc lập từng hàm nguồn và hàm chìm tương ứng của nó [18]. Ngoài ra, các hàm riêng lẻ có thể chứa nhiều điểm gọi nhảy cảm và các phương pháp truyền thống phân tích từng điểm gọi riêng biệt không chỉ lãng phí tài nguyên tính toán mà còn làm giảm đáng kể hiệu quả phân tích tổng thể [19].

Một hạn chế đáng kể khác của phân tích vết bản truyền thống bắt nguồn từ việc nó phụ thuộc vào các quy tắc tĩnh. Các phương pháp tiếp cận hiện tại, chẳng hạn như SaTC, Karonte và EmTaint, chủ yếu phụ thuộc vào các quy tắc lan truyền vết bản được xác định trước và các kỹ thuật khớp mẫu. Tuy nhiên, đối với các thiết bị IoT có kiến trúc đa dạng và tương tác thành phần phức tạp, các quy tắc tĩnh thường không bao gồm được tất cả các đường lan truyền vết bản có thể xảy ra, dẫn đến cả tỷ lệ dương tính giả và âm tính giả cao [20].

Để giải quyết những hạn chế này, chúng tôi đề xuất Liva, một hệ thống phân tích vết bản do LLM điều khiển được thiết kế đặc biệt để phát hiện lỗ hổng phần mềm IoT và chúng tôi đã triển khai một nguyên mẫu bao gồm hơn 9.000 dòng mã, tích hợp IDA Pro [21], Ghidra [22] và các tác nhân LLM chuyên dụng. Liva khắc phục những hạn chế của các phương pháp phân tích vết bản truyền thống bằng cách tận dụng các mô hình ngôn ngữ lớn đã được tinh chỉnh để hiểu mã ngữ nghĩa đa sắc thái và sử dụng cơ sở dữ liệu đồ thị để cho phép lưu trữ và truy vấn hiệu quả các luồng vết bản. Liva giới thiệu ba cải tiến quan trọng để giải quyết những thách thức nói trên:

Nhận dạng nguồn và phân chìm nâng cao: chúng tôi kết hợp thuật toán nhận dạng nâng cao để phân tích các chuỗi chỉ phân sụn để trích xuất tên hàm và mã dịch ngược gọi chúng. Sau đó, Liva sử dụng LLM để xác định chức năng nào có khả năng nhập dữ liệu của người dùng, phân loại chúng thành điểm nguồn. Đối với các điểm chìm nguy hiểm trong các thư viện dùng chung, Liva thực hiện phân tích sâu hơn về các phương pháp đóng gói của chúng, tạo điều kiện cho chiến lược nhận dạng toàn diện có thể áp dụng trong nhiều tình huống khác nhau.

Tối ưu hóa hiệu quả dựa trên phân cụm: chúng tôi đề xuất một phương pháp mới để phân tích các luồng chức năng nguy hiểm dựa trên phân cụm nguồn và chức năng nhạy cảm. Cách tiếp cận này nhóm các hàm nguồn có chung chức năng xử lý thành các cụm. Đối với mỗi cụm hàm nguồn, Liva sử dụng chiến lược khám phá thống nhất để duyệt qua biểu đồ lệnh gọi hàm bằng thuật toán tìm kiếm, bắt đầu từ tất cả các hàm nguồn và tạo ra một tập hợp các đường dẫn lệnh gọi dẫn đến các điểm chìm tiềm năng. Trong quá trình phân tích vết bản, các hàm trong cùng một chuỗi lệnh gọi hàm có chung cách gọi sẽ được nhóm lại với nhau để phân tích lan truyền dữ liệu vết bản thống nhất.

Hệ thống LLM đa tác nhân: chúng tôi giới thiệu Hệ thống phân tích vết như đa tác nhân mới dựa trên các mô hình ngôn ngữ lớn đã được tinh chỉnh. Bằng cách triển khai tác nhân lan truyền vết bản và tác nhân suy luận để bị tổn thương, Liva khắc phục các hạn chế của các quy tắc tĩnh truyền thống. LLM của tác nhân lan truyền vết bản, được tinh chỉnh theo kiến thức dành riêng cho miền, không chỉ có hiểu biết sâu sắc về ngữ nghĩa mã mà còn thích ứng linh hoạt với các kiểu lan truyền vết bản đa dạng và các mối quan hệ truyền tham số. Sau đó, tác nhân suy luận lỗ hổng sẽ tận dụng các kết quả phân tích này để xác định sự hiện diện của các rủi ro bảo mật thực tế, từ đó cải thiện độ chính xác đồng thời giảm thiểu các xác định sai.

Để đánh giá tính hiệu quả của Liva, chúng tôi đã tiến hành các thử nghiệm toàn diện bằng cách sử dụng bộ dữ liệu bao gồm 64 mẫu chương trình cơ sở được thu thập từ 11 nhà cung cấp IoT nổi bật. Tập dữ liệu này được xây dựng bằng cách kết hợp các bộ dữ liệu SaTC và Karonte đã thiết lập với 15 mẫu chương trình cơ sở mới bổ sung. Sau đó, việc đo điểm chuẩn mở rộng đã được tiến hành dựa trên các công cụ phân tích chương trình cơ sở tiên tiến, cụ thể là Karonte và SaTC, sử dụng bộ dữ liệu toàn diện gồm 64 thiết bị này. Các kết quả đánh giá nhấn mạnh khả năng phát hiện lỗ hổng vượt trội của Liva, thể hiện khả năng xác định thêm 309 lỗ hổng đã biết so với SaTC và nhiều hơn 349 lỗ hổng so với Karonte. Hơn nữa, Liva thể hiện độ tin cậy được nâng cao, đạt được mức giảm tỷ lệ dương tính giả lần lượt là 59,4% và 67,6% so với các đường cơ sở này. Hệ thống duy trì các số liệu hiệu suất mạnh mẽ, đạt tỷ lệ thu hồi 98,1% và độ chính xác 74,6%, đồng thời mang lại hiệu quả vượt trội, hoàn thành phân tích nhanh hơn 6,7 lần so với đường cơ sở hoạt động tốt nhất. Ngoài việc phát hiện các lỗ hổng đã biết, quá trình xác thực trong thế giới thực của Liva đã tiết lộ 64 lỗ hổng bảo mật chưa được tiết lộ trước đây, trong đó 39 lỗ hổng đã nhận được chỉ định CVE/CNVD chính thức thành công.

Những đóng góp chính của Liva được tóm tắt như sau: • Chúng tôi đề xuất một phương pháp có sự hỗ trợ của LLM để xác định chính xác các nguồn và nơi chứa vết bản. • Chúng tôi sử dụng tối ưu hóa dựa trên phân cụm, giúp tăng cường đáng kể hiệu quả trong phân tích lan truyền vết bản. • Chúng tôi là những người đầu tiên tận dụng cả LLM thương mại và tính chính xác để xây dựng một hệ thống đa tác nhân dành riêng cho phân tích lan truyền vết bản và phát hiện lỗ hổng sau đó. • Chúng tôi trình bày đánh giá toàn diện về Liva bằng cách sử dụng 64 bộ dữ liệu chương trình cơ sở, phát hiện 370 lỗ hổng trong đó có 64 lỗ hổng chưa được biết trước đây, với 39 lỗ hổng được báo cáo và chấp nhận chính thức.

Chúng tôi công bố công khai tập dữ liệu phân sụn, tài liệu chi tiết, mã nguồn và bộ sưu tập các ví dụ tinh chỉnh tại <https://github.com/RingBDStack/LIVA>.

II. LÝ LỊCH

A. Mô hình mối đe dọa

Nghiên cứu này điều tra các lỗ hổng liên quan đến vết bản trong các dịch vụ web của thiết bị nút mạng IoT, bao gồm bộ định tuyến, cổng VPN, điểm truy cập không dây, bộ lặp mạng và thiết bị lưu trữ mạng. Phân sụn trên các thiết bị này thường hoạt động trên nền tảng dựa trên Linux và các dịch vụ web phụ trợ của chúng thường được hỗ trợ bởi các chương trình nhúng [23].

Mô hình mối đe dọa của chúng tôi giả định rằng kẻ tấn công có thể lấy được chương trình cơ sở của thiết bị mục tiêu và thực hiện kỹ thuật đảo ngược để hiểu chi tiết triển khai các dịch vụ phụ trợ. Kẻ tấn công có thể truy cập dịch vụ web của thiết bị qua mạng LAN hoặc Internet và gửi các yêu cầu HTTP độc hại [24]. Các dịch vụ web IoT thường cung cấp nhiều điểm đầu vào, chẳng hạn như biểu mẫu đăng nhập, trang cấu hình và chức năng tải tệp lên. Chúng tôi coi các điểm truy cập này là nguồn có dấu vết tiềm ẩn có thể đưa ra dữ liệu không đáng tin cậy, có thể truyền đến các chức năng phụ trợ không an toàn và gây ra

vulnerabilities.

B. Phân tích vết bản

Phân tích vết bản là một kỹ thuật phân tích bảo mật chương trình quan trọng được sử dụng để theo dõi đường truyền của dữ liệu nguy hiểm tiềm tàng, cho phép xác định và ngăn chặn các lỗ hổng bảo mật. Quá trình phân tích vết bản thường bao gồm ba giai đoạn chính: xác định nguồn vết bản, vị trí chìm nhạy cảm và phân tích sự lan truyền vết bản. Tuy nhiên, các giải pháp hiện tại vẫn gặp phải một số thách thức.

Nhận dạng nguồn: Quá trình này nhằm mục đích xác định các chức năng hoặc điểm vận hành trong hệ thống nhận đầu vào bên ngoài, có khả năng bị kẻ tấn công thao túng và dẫn đến các lỗ hổng bảo mật. Trong các dịch vụ web của thiết bị IoT, thông tin đầu vào như vậy thường được lấy từ người dùng thông qua giao thức HTTP. SATC [10] sử dụng các từ khóa đầu vào phổ biến giữa front-end và back-end để xác định các nguồn taint, nhưng không thể phát hiện các nguồn thiếu mã frontend. LARA [25] mô tả chính xác các nguồn bị nhiễm độc bằng cách sử dụng URI và khóa nhưng không xác định được nguồn trong các phương pháp định tuyến đặc biệt của thiết bị IoT.

Nhận dạng phần chìm: Điều này đề cập đến việc xác định các điểm gọi chức năng nguy hiểm thực hiện các hoạt động nhạy cảm, chẳng hạn như thao tác chuỗi (ví dụ: `strcpy`, `strcat`), thực thi lệnh (ví dụ: `system`, `exec1`) và đầu ra được định dạng (ví dụ: `printf`, `fprintf`). SATC và Karonte sử dụng phần chìm được xác định trước nhưng không thể phân tích phần chìm trong thư viện dùng chung (.so) hoặc hỗ trợ phân tích phần chìm gia tăng.

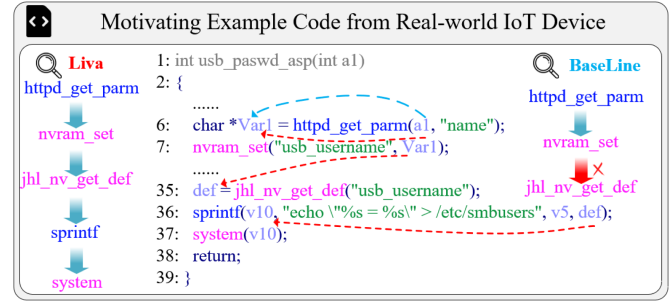
Phân tích truyền bá dữ liệu bị nhiễm độc: Phân tích bị nhiễm độc theo dõi quá trình truyền bá dữ liệu có khả năng không đáng tin cậy từ các điểm nguồn thông qua chương trình để xác định liệu cuối cùng dữ liệu đó có đạt đến các điểm chìm nhạy cảm hay không. LATTE [26] là nghiên cứu đầu tiên áp dụng LLM để phân tích vết bản. Nó tận dụng việc giải thích các đoạn mã nhị phân dựa trên lời nhắc để đánh giá khả năng kết nối của dữ liệu bị nhiễm độc. Tuy nhiên, LATTE không sử dụng lại các kết quả phân tích trung gian, dẫn đến tăng chi phí tính toán.

C. Mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn (LLM) là các kiến trúc mạng thần kinh tiên tiến được đào tạo trên các bộ sưu tập văn bản và mã lớn. Họ có thể nắm bắt được cú pháp, ngữ nghĩa và các mẫu ngữ cảnh phức tạp [27]. Bằng cách sử dụng đào tạo trước trên các bộ dữ liệu dành riêng cho miền, sau đó là tinh chỉnh, LLM đạt được hiệu suất mạnh mẽ trong việc hiểu ngôn ngữ tự nhiên, suy luận logic và phân tích ngữ nghĩa mã [28]. Trong phân tích bảo mật phần mềm, LLM có thể giải thích ngữ nghĩa chương trình cấp thấp, suy ra các mối quan hệ luồng dữ liệu và xác định các mẫu lỗ hổng tiềm ẩn [29]. Khả năng khái quát hóa trên các kiến trúc tập lệnh khác nhau và các kiểu mã hóa đa dạng khiến chúng đặc biệt phù hợp để phân tích vết bản nhị phân [30].

D. Động lực

Hình 1 hiển thị một đoạn mã chức năng trong phần sụn nhị phân của bộ định tuyến D-Link có chứa lỗ hổng chèn lệnh. Quá trình kích hoạt như sau: sau khi nhận được yêu cầu HTTP, tệp nhị phân phụ trợ `jhttpd` sử dụng so khớp tuyến để hướng luồng chương trình tới hàm `usb_passwd_asp`. Đầu tiên, hàm này đọc giá trị `name` từ yêu cầu HTTP



Hình 1. Ví dụ tạo động lực.

và gọi `nvram_set` để lưu trữ nó trong cấu hình. Sau đó, ở dòng 35, nó gọi `jhl_nv_get_def` để đọc lại giá trị `name` từ cấu hình. Ở dòng 36, nó gán giá trị cho biến `v10` bằng cách sử dụng `sprintf` và cuối cùng, ở dòng 37, nó thực thi `system` để kích hoạt việc chèn lệnh.

Trong thử nghiệm phát hiện lỗ hổng bảo mật, chúng tôi đã sử dụng SATC, KARONTE và EmTaint để phân tích vết bản. Không ai trong số họ có thể phát hiện chuỗi truyền tham số hoàn chỉnh. Lý do là các công cụ này dựa trên các quy tắc truyền biến cố định, không thể phân tích đầy đủ cách các biến được xử lý, nơi chúng được lưu trữ và cách chúng được truy xuất trong mã sau này. Các phương pháp phân tích dựa trên quy tắc tĩnh truyền thông như vậy hoạt động kém khi đường truyền lan truyền vết bản phức tạp.

Như được hiển thị trong Hình 1, phía bên phải trình bày kết quả từ các công cụ truyền thông, trong đó quá trình phân tích dừng lại khi biến được chuyển đến `nvram_set`, vì các công cụ này không thể diễn giải chức năng thực tế của nó và do đó không thể tiếp tục truy tìm.

Ngược lại, phía bên trái hiển thị phân tích của Liva, bằng cách nhận ra mối quan hệ ngữ nghĩa “lưu trữ-truy xuất” giữa `nvram_set` và `jhl_nv_get_def`, có thể liên kết tham số `def` thu được bởi `jhl_nv_get_def` với `var1` trước đó, từ đó phát hiện ra lỗ hổng.

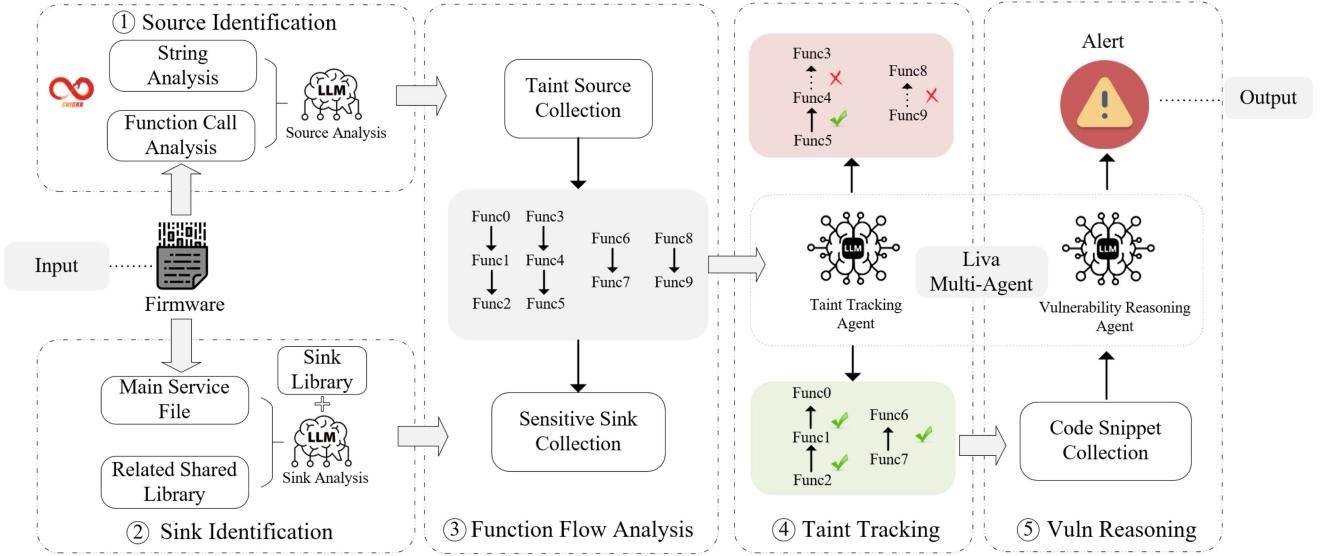
III. THỬ THÁCH

A. Nhận dạng nguồn và đích chính xác

Phần mềm IoT đặt ra những thách thức do kiến trúc phần cứng đa dạng (ARM, MIPS, x86) và cơ chế xử lý đầu vào không đồng nhất [31]. Nhận dạng nguồn/sink truyền thông do chuyên gia xác định thiếu khả năng mở rộng trên các biến thể chương trình cơ sở, trong khi các phương pháp đối sánh chuỗi giao diện người dùng-phụ trợ phải chịu tỷ lệ dương tính giả cao do phân tích cú pháp không rõ ràng.

B. Phân tích vết bản hiệu quả

Các tệp nhị phân của chương trình cơ sở IoT chứa các biểu đồ gọi hàm mở rộng với sự chồng chéo đường dẫn đáng kể, trong đó nhiều vùng dễ bị tổn thương chia sẻ các chức năng xử lý trung gian chung [32]. Phân tích vết bản ngây thơ đưa ra sự dư thừa tính toán thông qua việc lặp đi lặp lại các đường dẫn mã giống hệt nhau. Thách thức chính là phát triển các chiến lược phân cụm để hợp nhất các nguồn, phần chìm và chức năng trung gian có liên quan, từ đó giảm thiểu phân tích dư thừa trong khi vẫn duy trì độ chính xác của việc phát hiện.



Hình 2. Tổng quan về Liva.

C. Những thách thức khi ứng dụng tuyến LLM

Việc áp dụng các mô hình ngôn ngữ lớn để phân tích vết bản trong phần mềm IoT đặt ra hai thách thức quan trọng. Đầu tiên, các giới hạn về mã thông báo hạn chế việc phân tích mã dịch ngược quy mô lớn, thường yêu cầu cắt bớt làm ảnh hưởng đến tính đầy đủ của phân tích [33], [34]. Vấn đề này đặc biệt rõ ràng khi phân tích phần sụn phức tạp được đặc trưng bởi biểu đồ lệnh gọi chức năng mở rộng và cấu trúc lồng sâu. Thứ hai, việc đạt được các định dạng đầu ra được tiêu chuẩn hóa phù hợp với quá trình xử lý tự động đòi hỏi phải tinh chỉnh theo miền cụ thể thay vì chỉ sử dụng kỹ thuật nhắc nhở [35]. Sự thay đổi trong các phản hồi LLM gây ra sự không nhất quán trong phân tích cú pháp làm giảm độ tin cậy của phân tích tiếp theo. Việc xây dựng các tập dữ liệu huấn luyện chất lượng cao để nắm bắt chính xác ngữ nghĩa lan truyền vết bản trong quá trình triển khai phần mềm cơ sở khác nhau vẫn là một thách thức cơ bản đối với việc áp dụng hiệu quả LLM trong phân tích vết bản. Các bộ dữ liệu như vậy phải bao gồm đầy đủ các kiểu truyền tham số và các biểu hiện dễ bị tổn thương gặp phải trong các hệ thống IoT trong thế giới thực.

IV. THIẾT KẾ CUỘC SỐNG

Để giải quyết những thách thức này, chúng tôi đề xuất Liva, một hệ thống dựa trên LLM để phân tích lỗ hổng IoT. Như được hiển thị trong Hình 2, Liva áp dụng kiến trúc mô-đun để chuyển đổi các tệp nhị phân của chương trình cơ sở thành báo cáo lỗ hổng qua năm giai đoạn: 1) Nhận dạng nguồn: Giai đoạn này kết hợp nhận dạng hàm chuỗi với phân tích ngữ nghĩa do LLM cung cấp để xác định chính xác các điểm đầu vào bên ngoài. 2) Nhận dạng phần chìm: Ở đây, các thư viện lỗ hổng được xác định trước được tích hợp với phân tích dựa trên LLM để xác định các chức năng bao bọc quan trọng về bảo mật đại diện cho các phần chìm lỗ hổng tiềm ẩn. 3) Phân tích luồng chức năng nguy hiểm: Kỹ thuật cắt mã được áp dụng để thiết lập các kết nối cấp chức năng giữa các nguồn ứng cử viên được xác định và phần chìm. 4) Theo dõi sự lan truyền của vết bản: Một tác nhân Liva được tinh chỉnh được sử dụng để theo dõi các phụ thuộc dữ liệu và xác định đường truyền lan truyền

từ nguồn tới bồn chứa. 5) Suy luận về lỗ hổng: Cuối cùng, các đoạn mã và siêu dữ liệu được kết nối được phân tích bởi một tác nhân suy luận để phân loại các lỗ hổng và đánh giá mức độ nghiêm trọng của chúng.

A. Nhận dạng nguồn

Để tự động xác định các nguồn đầu vào trong phần mềm IoT, chúng tôi đề xuất một phương pháp dựa trên phụ trợ tận dụng các chuỗi không đổi trong các tệp nhị phân kết hợp với khả năng phân tích ngữ nghĩa của LLM. Không giống như các phương pháp tiếp cận truyền thống dựa vào mã giao diện người dùng, phương pháp của chúng tôi phân tích trực tiếp các tệp nhị phân phía sau, được chứng minh là đặc biệt hiệu quả trong các tình huống trong đó sự liên kết giữa giao diện người dùng và mặt sau yếu hoặc khi không có tệp giao diện người dùng.

Phương pháp này hoạt động theo ba giai đoạn. 1) *Trích xuất chuỗi*: các chuỗi hằng $S = \{s_1, s_2, \dots, s_n\}$ được trích xuất từ các phần nhị phân (ví dụ: `.rodata`). 2) *Bản địa hóa tham chiếu*: đối với mỗi chuỗi $s_i \in S$, chúng tôi định vị các tham chiếu của nó trong tệp nhị phân và xác định các lệnh gọi hàm tương ứng trong đó s_i xuất hiện dưới dạng đối số. Đặt tập hợp ứng cử viên kết quả là F . 3) *Phân loại dựa trên LLM*: mỗi $f \in F$ được LLM phân tích để xác định xem liệu cuộc gọi có thể hiện một cách ngữ nghĩa một hoạt động đầu vào bên ngoài hay không (ví dụ: truy xuất tham số HTTP).

Do đó, tập hợp cuối cùng của các hàm nguồn là:

$$\mathcal{F}_{\text{source}} = \{f \in F \mid \text{LLM_classify}(f) = 1\} \quad (1)$$

B. Nhận dạng bồn rửa

Để phát hiện toàn diện các lệnh gọi hàm nguy hiểm tiềm tàng (điểm chìm), chúng tôi thiết kế một khung cấp độ kép để phân tích cả *chương trình chính* và *thư viện dùng chung*. Chiến lược này cho phép chúng tôi nắm bắt cả rủi ro rõ ràng và rủi ro được gói gọn về mặt ngữ nghĩa do các chức năng thư viện đưa ra một cách gián tiếp.

1) *Cấp độ chương trình chính* (So khớp chức năng trực tiếp): Trong chương trình chính, các lời gọi hàm nguy hiểm tiềm tàng là

được xác định thông qua so sánh trực tiếp với một tập hợp bồn chứa nguy hiểm được xác định trước, ký hiệu là $\mathcal{D}_{\text{sink}}$ (ví dụ: `system`, `popen`, `str-cpy`). Bất kỳ lệnh gọi hàm nào khớp chính xác với mục nhập trong $\mathcal{D}_{\text{sink}}$ ngay lập tức được phân loại là phần chìm, do đó cho phép phát hiện nhanh chóng và chính xác các trang web cuộc gọi dễ bị tổn thương.

2) *Cấp độ thư viện dùng chung (Nhận dạng chức năng trình bao bọc)*: Phần sụn IoT sử dụng rộng rãi các thư viện dùng chung có các chức năng được xuất thường bao gồm các hoạt động nguy hiểm. Chúng tôi sử dụng phương pháp tiếp cận hai giai đoạn để phát hiện những rủi ro gián tiếp này.

Đầu tiên, chúng tôi thu thập tập hợp các hàm được xuất từ tất cả các thư viện dùng chung được chương trình chính tham chiếu và áp dụng các phương pháp chẩn đoán nhẹ—chẳng hạn như các hàm liên quan đến đối số con trỏ hoặc bộ đệm, chứa các lệnh gọi lồng nhau hoặc bao gồm các từ khóa đáng ngờ như `exec`, `auth` hoặc `parse`—để lọc một tập hợp con ứng cử viên. Thứ hai, đối với mỗi hàm ứng viên l , chúng ta xây dựng biểu đồ luồng điều khiển của nó và thực hiện lan truyền vết bản bằng LLM. Nếu phân tích vết bản cho thấy đường dẫn từ l đến bất kỳ phần chìm nguy hiểm nào đã biết $d \in \mathcal{D}_{\text{sink}}$ thì l được phân loại là hàm chìm bao bọc. Cơ chế này cho phép chúng tôi xác định các điểm chìm gián tiếp không thể nhìn thấy ở cấp độ chương trình chính nhưng được hiển thị thông qua việc trừu tượng hóa thư viện dùng chung.

3) *Nhận dạng phần chìm kết hợp*: Bộ phần chìm cuối cùng $\mathcal{F}_{\text{sink}}$ kết hợp các phần chìm trực tiếp từ chương trình chính và các phần chìm bao bọc từ các thư viện dùng chung, đóng vai trò là nền tảng cho việc lan truyền vết bản và phát hiện lỗ hổng.

C. Tối ưu hóa phân tích vết bản

1) *Phân tích luồng chức năng dựa trên phân cụm nguồn*: Để cải thiện hiệu quả và phạm vi phân tích vết bản liên tục, chúng tôi đề xuất phương pháp phân tích luồng chức năng từ trên xuống dựa trên *phân cụm nguồn*. Bằng cách nhóm các hàm nguồn có liên quan thành các cụm và cùng khám phá các đường dẫn cuộc gọi của chúng, việc truyền tải dư thừa sẽ được giảm bớt trong khi vẫn đảm bảo phạm vi bao phủ toàn diện cho các điểm chìm tiềm năng.

Đặt $\mathcal{F}_{\text{nguồn}}$ biểu thị tập hợp đầy đủ các hàm nguồn được xác định. Trong khung của chúng tôi, các nguồn này được nhóm lại thông qua thủ tục $\text{CLUSTER}(\mathcal{F}_{\text{nguồn}})$, nhóm này nhóm các hàm nguồn theo các hàm gốc của chúng trong biểu đồ lệnh gọi. Đối với mỗi hàm nguồn f_s , chúng ta lấy hàm mẹ $\text{parent}(f_s) = g$ trong đó $(g \rightarrow f_s) \in E$, nghĩa là g là hàm gọi trực tiếp của f_s . Sau đó, quy trình CLUSTER tổng hợp tất cả các hàm nguồn chia sẻ cùng hàm cha vào một cụm duy nhất, được biểu thị dưới dạng $C(g) = \{f_s \mid \text{parent}(f_s) = g\}$. Tập hợp cụm hoàn chỉnh được CLUSTER trả về chỉ đơn giản là tập hợp của tất cả $C(g)$ (không trống). Việc nhóm dựa trên nguồn gốc này đảm bảo rằng các nguồn bắt nguồn từ cùng một chức năng kèm theo sẽ được phân tích chung, do đó giảm việc truyền tải dư thừa trên các đường dẫn cuộc gọi tương tự.

Chúng ta mô hình hóa chương trình dưới dạng biểu đồ cuộc gọi $G = (\mathcal{F}, E)$, trong đó $\mathcal{F}$ là tập hợp các hàm và E tập hợp các cạnh cuộc gọi. Đối với mỗi cụm $C_i \in \mathcal{C}_{\text{nguồn}}$, chúng tôi thực hiện khám phá chung từ trên xuống từ tất cả các nguồn trong C_i . Nếu một đường dẫn từ C_i đạt đến chức năng đích, thì đường dẫn đó được ghi là có thể truy cập được. Về mặt hình thức, tập hợp các bồn chứa được phát hiện từ cụm C_i là

$$\mathcal{F}_{\text{sink}}^{(i)} = \left\{ f^{(t)} \in \mathcal{F}_{\text{sink}} \mid \exists \text{ path from } C_i \text{ to } f^{(t)} \right\} \quad (2)$$

Algorithm 1: Cluster-Based Taint Flow Discovery.

Require: Call graph $G = (\mathcal{F}, E)$; source set $\mathcal{F}_{\text{source}}$; sink set $\mathcal{F}_{\text{sink}}$

Ensure: Dangerous flow set $\mathcal{P}_{\text{danger}}$

```

1: function DISCOVERFLOWS( $G, \mathcal{F}_{\text{source}}, \mathcal{F}_{\text{sink}}$ )
2:    $\mathcal{C} \leftarrow \text{CLUSTER}(\mathcal{F}_{\text{source}})$ 
3:    $\mathcal{P}_{\text{danger}} \leftarrow \emptyset$ 
4:   for all  $C \in \mathcal{C}$  do
5:     for all  $f \in C$  do
6:        $\mathcal{P}_f \leftarrow \text{FINDPATHS}(f, \mathcal{F}_{\text{sink}}, G)$ 
7:        $\mathcal{P}_{\text{danger}} \leftarrow \mathcal{P}_{\text{danger}} \cup \mathcal{P}_f$ 
8:     end for
9:   end for
10:  return  $\mathcal{P}_{\text{danger}}$ 
11: end Function

```

Nói cách khác, một lần truyền tải bắt đầu từ một cụm nguồn có thể bao gồm nhiều đường dẫn cuộc gọi và phần đích, do đó tránh được việc thăm dò dư thừa trên mỗi nguồn. Thuật toán 1 tóm tắt chiến lược dựa trên cụm. Bằng cách chia sẻ quá trình truyền tải giữa các nguồn có liên quan về mặt ngữ nghĩa, phương pháp này cải thiện khả năng mở rộng phân tích vết bản liên tục trong phần mềm IoT quy mô lớn.

2) *Cấu trúc ánh xạ tham số*: Để giải quyết thách thức cơ bản trong việc lập mô hình chính xác các phụ thuộc dữ liệu giữa các thủ tục trong phân tích chương trình tĩnh, chúng tôi giới thiệu một cấu trúc ánh xạ tham số mới có khả năng nắm bắt chính thức ngữ nghĩa của các mối quan hệ lệnh gọi hàm. Không giống như các cách tiếp cận truyền thống dựa trên các biểu đồ luồng điều khiển phức tạp, cấu trúc của chúng tôi cung cấp một sự trừu tượng tập trung mô hình hóa ngữ nghĩa truyền tham số trong khi vẫn duy trì hiệu quả.

Cách tiếp cận của chúng tôi thể hiện mỗi mối quan hệ gọi hàm dưới dạng một bộ dữ liệu có cấu trúc $\langle f_c, \mathcal{M}, a, c \rangle$ được liên kết với hàm cha f_p , trong đó $f_c \in \mathcal{F}$ biểu thị hàm được gọi, $\mathcal{M} : \mathcal{V}_{f_p} \rightarrow \mathcal{P}_{f_c}$ biểu thị ánh xạ tham số từ biến cha đến tham số con, a chỉ định địa chỉ trang gọi và c chứa mã dịch ngược tương ứng mảnh vỡ. Về mặt hình thức, cấu trúc dữ liệu hoàn chỉnh của chúng tôi có thể được biểu diễn dưới dạng:

$$\mathcal{D} = \{f_p \mapsto \langle f_c, \mathcal{M}, a, c \rangle \mid f_p \in \mathcal{F}_{\text{parent}}, f_c \in \mathcal{F}_{\text{child}}\} \quad (3)$$

trong đó hàm ánh xạ $\mathcal{M} = \{v_p \mapsto p_c \mid v_p \in \mathcal{V}_{f_p}, p_c \in \mathcal{P}_{f_c}\}$ thiết lập sự tương ứng chính xác giữa các biến gọi $\mathcal{V}_{f_p}$ và các tham số callee $\mathcal{P}_{f_c}$. Để minh họa cụ thể cấu trúc này, việc ánh xạ tham số có thể được biểu diễn bằng ký hiệu đơn giản sau:

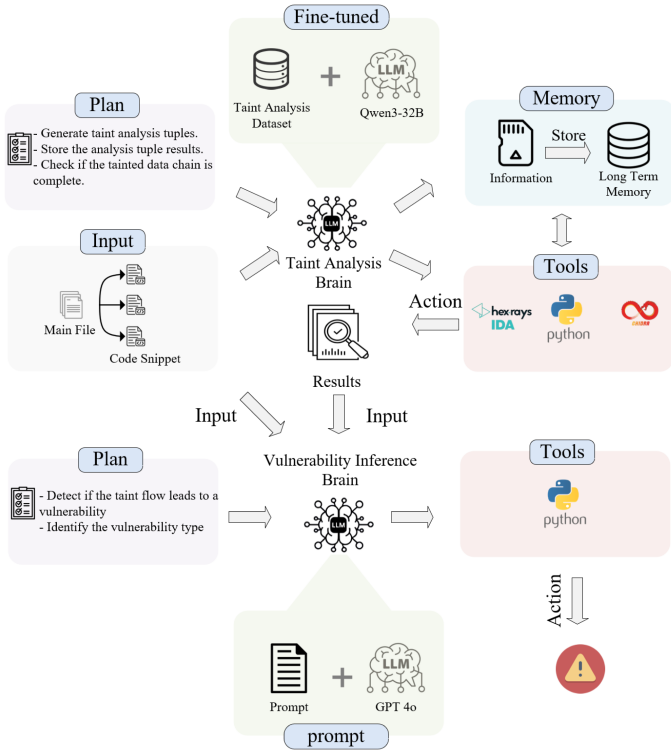
```

{ ParentFunction:
  (CalledFunction, ParamMapping, CallAddress, CallCode) }

```

Cấu trúc mối quan hệ truyền tham số này $\mathcal{M}$ bảo toàn hoàn toàn thông tin đường dẫn luồng dữ liệu, từ đó cho phép phân tích lan truyền vết bản toàn diện. Quan trọng nhất, thuộc tính *thành phần phân cấp* cho phép cấu trúc của chúng ta xử lý một cách tự nhiên các chuỗi cuộc gọi phức tạp thông qua ứng dụng đệ quy: đối với chuỗi cuộc gọi $f_1 \rightarrow f_2 \rightarrow \dots \rightarrow f_n$, luồng tham số hoàn chỉnh có thể được tính như sau:

$$\mathcal{M}_{1,n} = \mathcal{M}_{n-1,n} \circ \mathcal{M}_{n-2,n-1} \circ \dots \circ \mathcal{M}_{1,2} \quad (4)$$



Hình 3. Khung đa tác nhân Liva. Khung này bao gồm hai thành phần chính: tác nhân phân tích vết bản và tác nhân suy luận lỗ hổng.

Thuộc tính kết hợp này cung cấp những lợi thế đáng kể cho việc phân tích chương trình quy mô lớn. Khả năng tái sử dụng của các nút trung gian cho phép truy cập thường xuyên vào ánh xạ tham số hàm $\mathcal{M}_{i,j}$, do đó tránh được việc phân tích đường dẫn phụ dư thừa và cho phép phân tích đồng thời.

3) *Hợp nhất chuỗi cuộc gọi*: Một nút thất hiệu quả nghiêm trọng trong phân tích liên thủ tục truyền thống phát sinh từ việc xử lý dư thừa các hàm cha mẹ gọi nhiều hàm con. Để giải quyết vấn đề tính toán kém hiệu quả này, chúng tôi giới thiệu *chiến lược hợp nhất địa điểm cuộc gọi* để cùng có bối cảnh phân tích và giảm đáng kể chi phí phân tích.

Hãy xem xét hàm cha $f_p \in \mathcal{F}_{\text{parent}}$ gọi nhiều hàm con trên các trang gọi khác nhau. Chúng tôi chính thức xác định ngữ cảnh gọi của nó là $\mathcal{C}(f_p) = \{(f_c, l) \mid f_c \in \mathcal{F}_{\text{child}}, l \in \mathcal{L}_{f_p}\}$, trong đó mỗi bộ dữ liệu (f_c, l) đại diện cho một lệnh gọi hàm cụ thể tới f_c tại vị trí l trong không gian địa chỉ của hàm cha $\mathcal{L}_{f_p}$. Các phương pháp tiếp cận truyền thống phân tích từng mối quan hệ cha-con (f_p, f_c) một cách độc lập, dẫn đến $|\mathcal{C}(f_p)|$ các lần duyệt riêng biệt của cùng một mã hàm cha, dẫn đến sự dư thừa tính toán đáng kể.

Chiến lược tối ưu hóa của chúng tôi loại bỏ sự dư thừa này thông qua khung phân tích đồng nhất. Thay vì thực hiện các phân tích riêng lẻ, chúng tôi xây dựng bối cảnh phân tích hợp nhất:

$$\mathcal{M}(f_p) = \bigcup_{(f_c, l) \in \mathcal{C}(f_p)} \text{AnalyzeCall}(f_p, f_c, l) \quad (5)$$

trong đó hàm $\text{AnalyzeCall}(f_p, f_c, l)$ nắm bắt các mối quan hệ ánh xạ tham số và dòng lỗi cho các đối tượng cụ thể

gọi phiên bản từ f_p đến f_c tại vị trí l . Việc hợp nhất này cho phép duyệt qua toàn diện f_p đồng thời trích xuất tất cả các mối quan hệ cuộc gọi có liên quan, chuyển đổi nhiều phân tích độc lập thành một quy trình xử lý thống nhất.

Tác động thực tế của việc tối ưu hóa này đặc biệt rõ rệt trong các cơ sở mã chương trình cơ sở và hệ thống nhúng, nơi các cấu trúc cuộc gọi dày đặc và các mẫu lệnh gọi lặp đi lặp lại phổ biến. Trong quá trình triển khai của chúng tôi, phân tích hợp nhất này được tiếp tục kích hoạt thông qua mô hình ngôn ngữ lớn được tinh chỉnh, có khả năng diễn giải đồng thời nhiều trang web cuộc gọi và trích xuất tham số tương ứng của chúng và ngữ nghĩa lan truyền vết bản trong một lượt suy luận duy nhất. Bằng cách ủy quyền suy luận địa điểm cuộc gọi thống nhất cho mô hình, chúng tôi tránh đưa ra các truy vấn mô hình riêng biệt cho từng mối quan hệ cuộc gọi riêng lẻ, từ đó loại bỏ chi phí tương tác vốn có trong quy trình phân tích mỗi cuộc gọi truyền thống.

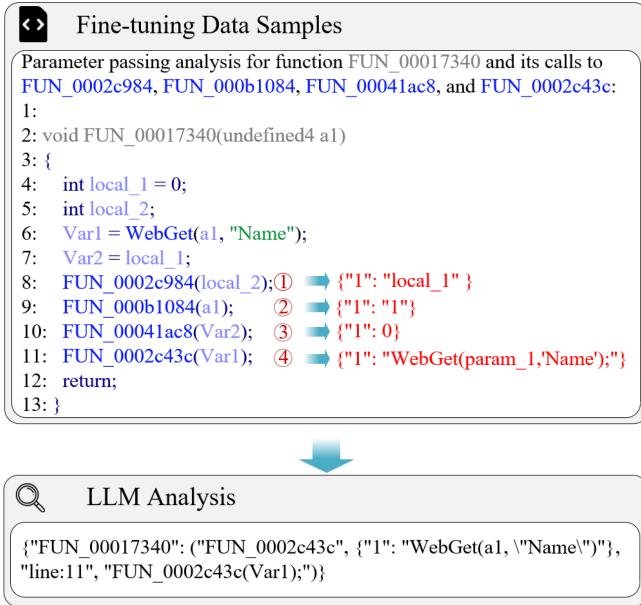
Do đó, việc tổng hợp nhiều phân tích trang web cuộc gọi vào bối cảnh thống nhất $\mathcal{M}(f_p)$ không chỉ loại bỏ việc truyền mã dư thừa mà còn giảm đáng kể số lượng tương tác phân tích mô hình cần thiết để xử lý các chức năng phần mềm phức tạp. Việc tích hợp việc hợp nhất trang web cuộc gọi với suy luận dựa trên LLM chung này cho phép tối ưu hóa cuộc gọi chéo có thể mở rộng hơn, chẳng hạn như suy luận biến được chia sẻ và theo dõi lan truyền vết bản hợp nhất trên nhiều đường dẫn gọi trong cùng một hàm gốc.

D. Liva Multi-Agent

Để phân tích khả năng kết nối dữ liệu bị nhiễm độc trong chuỗi lệnh gọi chức năng nguy hiểm và xác định chính xác các lỗ hổng bảo mật tiềm ẩn, chúng tôi đề xuất Liva, một hệ thống đa tác nhân mới bao gồm hai thành phần chuyên biệt: một tác nhân theo dõi luồng vết bản và một tác nhân suy luận để bị tổn thương. Tác nhân theo dõi luồng nhiễm độc sử dụng mô hình phụ thuộc biến đổi phức tạp để theo dõi một cách có hệ thống các đường truyền dữ liệu bị nhiễm độc từ các nguồn đến các điểm chương trình tùy ý. Thông qua phân tích luồng toàn diện, nó xây dựng các biểu đồ lan truyền vết bản chi tiết để nắm bắt cấu trúc liên kết luồng dữ liệu hoàn chỉnh trong chương trình đích. Dựa trên các biểu đồ này, tác nhân suy luận lỗ hổng sẽ tiến hành đánh giá rủi ro bảo mật bằng cách tận dụng thư viện các mẫu lỗ hổng được xác định trước để đánh giá một cách có hệ thống các điểm chìm quan trọng, bao gồm các chức năng thực thi lệnh hệ thống và giao diện thao tác bộ nhớ. Cách tiếp cận tích hợp này cho phép xác định lỗ hổng bảo mật có độ chính xác cao và hiệu quả trên các hệ thống phần mềm phức tạp.

1) *Tác nhân theo dõi vết bản*: Phần trên của Hình 3 minh họa quy trình làm việc của tác nhân này, sử dụng một mô hình ngôn ngữ lớn được tinh chỉnh với các ví dụ cụ thể về vết bản làm công cụ phân tích cốt lõi của nó. Theo lịch trình nhiệm vụ được xác định trước, tác nhân xử lý các đoạn chức năng và ghi lại các mối quan hệ truyền tham số trong cơ sở dữ liệu quan hệ. Những bản ghi này sau đó được truy vấn bằng các công cụ phân tích để xác định và xuất ra các luồng dữ liệu có vết bản được kết nối.

a) *Tinh chỉnh các mô hình ngôn ngữ lớn*: Để điều chỉnh các mô hình ngôn ngữ lớn cho các nhiệm vụ phân tích dòng chảy vết bản, chúng tôi sử dụng LLaMAFactory [36] làm khung tinh chỉnh của mình. Chúng tôi xây dựng các tập dữ liệu huấn luyện trong đó mỗi mẫu bao gồm các đoạn mã



Hình 4. Tinh chỉnh dữ liệu với bốn mối quan hệ ánh xạ tham số.

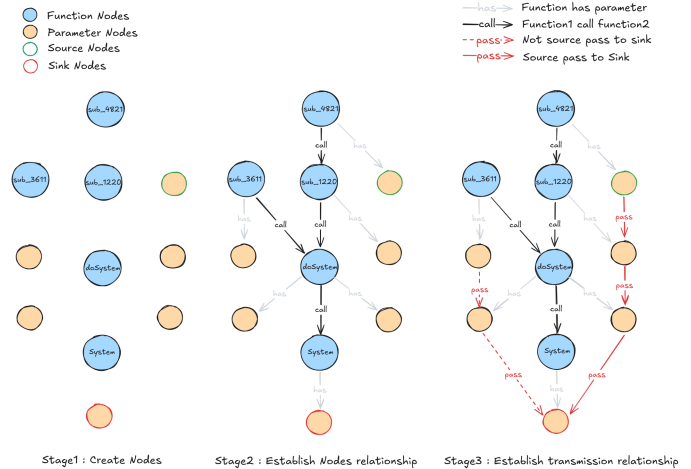
được chú thích bằng thông tin nguồn bị hư hỏng, kết hợp với các mẫu lan truyền vết bản tương ứng làm đầu ra mục tiêu.

Quá trình tinh chỉnh thúc đẩy việc triển khai QLoRA, kết hợp lượng tử hóa 4 bit với Thích ứng xếp hạng thấp để đạt được hiệu quả tính toán trong khi vẫn duy trì hiệu suất mô hình. Cách tiếp cận này cho phép điều chỉnh hiệu quả các mô hình ngôn ngữ được đào tạo trước để hiểu ngữ nghĩa lan truyền vết bản thông qua các chỉ dẫn lược tạo hiệu quả về tham số.

b) Tinh chỉnh mẫu dữ liệu: Để cho phép mô hình xử lý mã và giải thích phân tích của nó, các định dạng đầu vào và đầu ra đã được chuẩn hóa. Mỗi mẫu tinh chỉnh bao gồm một đầu vào được xây dựng và đầu ra tương ứng của nó, như minh họa trong Hình 4. Phần trên của hình mô tả đầu vào, xác định mục tiêu phân tích và cung cấp mã dịch ngược được chú thích bằng số dòng, trong khi phần dưới trình bày đầu ra được định dạng theo cấu trúc ánh xạ tham số được mô tả trong Phần IV-C2.

Thông qua phân tích có hệ thống các mẫu chương trình cơ sở của thiết bị trong thế giới thực, chúng tôi đã xác định được bốn loại cơ chế ánh xạ tham số riêng biệt: 1) lan truyền biến cục bộ, trong đó các tham số được truyền qua các phép gán biến cục bộ trong phạm vi chức năng; 2) truyền tham số chính thức, trong đó các tham số được truyền qua danh sách đối số hàm tuân theo các quy ước gọi tiêu chuẩn; 3) lan truyền giá trị không đổi, trong đó các tham số được truyền dưới dạng hằng số thời gian biên dịch hoặc thời gian chạy; và 4) lan truyền giá trị trả về, trong đó các tham số được truyền qua các giá trị trả về của hàm qua các ranh giới cuộc gọi. Như được đánh dấu màu đỏ trong Hình 4, ánh xạ của mô hình tương ứng trực tiếp với bốn loại này.

c) Lưu trữ và truy vấn các cấu trúc quan hệ: Chúng tôi sử dụng cơ sở dữ liệu đồ thị Neo4j [37] để lưu trữ các mối quan hệ ánh xạ tham số được xác định bởi tác nhân thông minh phân tích vết bản của chúng tôi. Lược đồ biểu đồ bao gồm hai loại nút: *Nút chức năng* biểu thị các hàm riêng lẻ với siêu dữ liệu của chúng,



Hình 5. Lưu trữ và truy vấn dữ liệu mờ. Lưu trữ bao gồm các nút chức năng, các nút tham số và các mối quan hệ của chúng. Truy vấn tập trung vào phân tích kết nối giữa nguồn và đích.

và *Nút tham số* biểu thị các tham số và biến trong phạm vi hàm. Mối quan hệ giữa các nút nắm bắt các kết nối ánh xạ tham số, trong đó mỗi cạnh biểu thị luồng dữ liệu từ tham số người gọi đến tham số callee.

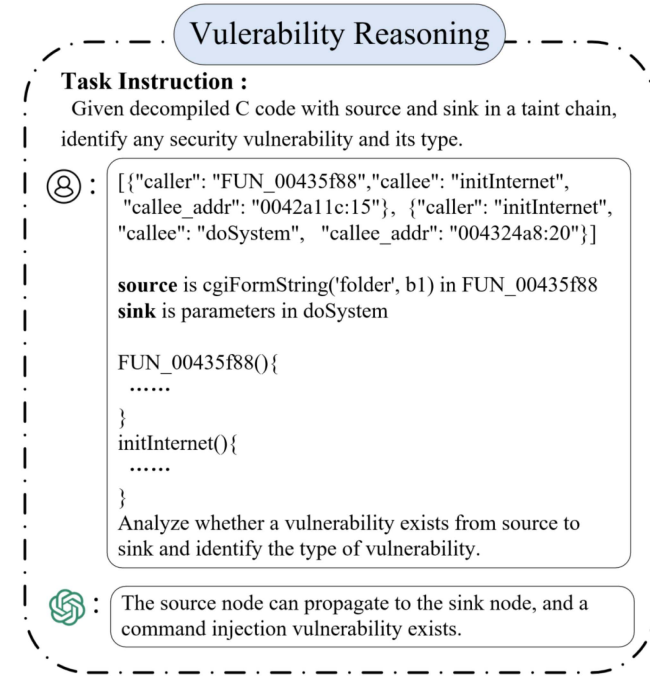
Hình 5 mô tả thủ tục lưu trữ, được tổ chức thành ba giai đoạn. Trong giai đoạn đầu tiên, các nút chức năng và nút tham số tương ứng được xây dựng dựa trên đầu ra của tác nhân Liva, đồng thời các nút nguồn và nút đích được xác định và gắn nhãn. Giai đoạn thứ hai thiết lập mối quan hệ gọi giữa các nút chức năng cũng như liên kết quyền sở hữu giữa các nút tham số và các chức năng tương ứng của chúng. Trong giai đoạn thứ ba, mỗi quan hệ truyền giữa các nút tham số được thêm vào để thể hiện đường dẫn luồng dữ liệu tổng thể.

Sau đó, khả năng kết nối của các luồng dữ liệu bị nhiễm độc có thể được truy vấn bằng cú pháp truy vấn Neo4j. Trong Hình 5, các mũi tên liên màu đỏ ở giai đoạn thứ ba biểu thị các luồng hợp lệ từ nguồn đến đích, trong khi các mũi tên nét đứt màu đỏ biểu thị các luồng bị nhiễm độc không hợp lệ trong đó điểm bắt đầu không phải là nút nguồn.

2) Tác nhân suy luận lỗ hổng bảo mật: Mặc dù tính năng theo dõi luồng vết bản xác định thành công các đường dẫn truyền dữ liệu tiềm năng từ nguồn đến nơi lưu trữ, nhưng sự tồn tại đơn thuần của các đường dẫn đó không đảm bảo sự hiện diện của các lỗ hổng có thể khai thác được. Đánh giá lỗ hổng nghiêm trọng yêu cầu xác định xem dữ liệu đầu vào có thể vượt quá ranh giới vùng đệm, bỏ qua các cơ chế xác thực hoặc phá vỡ các biện pháp kiểm soát bảo mật hay không. Nhiều luồng từ nguồn đến bồn chứa bảo vệ bằng cơ chế dọn dẹp, được giới hạn bởi các biện pháp bảo vệ có điều kiện hoặc không thể khai thác được thông qua các quy trình xác thực thích hợp. Để giải quyết sự khác biệt giữa các luồng dữ liệu lý thuyết và các mối đe dọa bảo mật thực tế, chúng tôi giới thiệu tác nhân suy luận lỗ hổng bằng cách sử dụng phân tích ngữ nghĩa để lọc các kết quả dương tính giả và xác nhận các lỗ hổng thực sự.

Như được minh họa trong Hình 6, tác nhân nhận đầu vào là chuỗi luồng vết bản được kết nối được xác định bởi giai đoạn theo dõi, cùng với các đoạn mã được dịch ngược tương ứng của tất cả các chức năng liên quan. Mỗi luồng ứng cử viên được biểu diễn dưới dạng một chuỗi có thứ tự

$$p = \langle f_s \rightarrow f_{i_1} \rightarrow f_{i_2} \rightarrow \dots \rightarrow f_t \rangle, \text{ trong đó}$$



Hình 6. Lời nhắc suy luận về lỗ hổng bảo mật.

f_s biểu thị hàm nguồn taint, f_t biểu thị hàm chìm và $f_{i,j}$ là các hàm trung gian trong đường truyền. Các mối quan hệ ảnh xạ tham số $\mathcal{M}_{i,j}$ được ghi lại trong quá trình theo dõi vết bản cung cấp sự tương ứng biến đổi chính xác giữa các ranh giới chức năng, cho phép phân tích ngữ nghĩa toàn diện của toàn bộ chuỗi luồng.

Để đánh giá một cách có hệ thống khả năng khai thác của từng đường dẫn ứng viên, chúng tôi sử dụng LLM thương mại với các lời nhắc được thiết kế cẩn thận để hướng dẫn suy luận của nó. Khung này chỉ đạo mô hình kiểm tra từng luồng để tìm các chỉ báo dễ bị tổn thương, bao gồm độ dài đầu vào không được kiểm tra, thiếu xác thực ranh giới và các lệnh gọi hàm nguy hiểm trong các đường dẫn quan trọng.

Suy luận thúc đẩy sự hiểu biết của mô hình về ngữ nghĩa lập trình và các nguyên tắc bảo mật để đánh giá xem mỗi đường dẫn ứng viên $p \in \mathcal{P}_{\text{candidate}}$ có phải là mối đe dọa bảo mật thực sự hay không. Chúng tôi xác định hàm phán đoán nhị phân $\mathcal{J}$: $\text{Pứng viên} \rightarrow 0, 1$ sao cho $\mathcal{J}(p) = 1$ chỉ ra lỗ hổng đã được xác nhận và $\mathcal{J}(p) = 0$ đại diện cho một dòng chảy lành tính.

Phương pháp suy luận ngữ nghĩa này trực tiếp giải quyết thách thức cốt lõi trong việc giảm thiểu các kết quả dương tính giả trong quá trình phát hiện lỗ hổng tĩnh. Các công cụ phân tích truyền thống không chỉ gán cờ các phân chìm nguy hiểm như `strcpy()` mà còn kết hợp các quy tắc heuristic để phát hiện xem ranh giới vùng đệm có được kiểm tra hay không. Tuy nhiên, những cách tiếp cận này không nắm bắt được bối cảnh ngữ nghĩa đầy đủ: việc kiểm tra ranh giới có thể tồn tại nhưng không thể đảm bảo an toàn trong luồng điều khiển phức tạp, sự phụ thuộc gián tiếp hoặc xác thực yếu. Ngược lại, tác nhân suy luận của chúng tôi tích hợp chuỗi cuộc gọi đầy đủ, ánh xạ biến và bảo vệ theo ngữ cảnh vào đánh giá. Điều này cho phép nó phân biệt các luồng thực sự có thể khai thác được – chẳng hạn như đầu vào không được kiểm tra dẫn tới lệnh gọi `system()` – với các luồng được bảo vệ bằng xác thực ranh giới hiệu quả. Do đó,

tập ứng viên $\mathcal{P}_{\text{ứng viên}}$ được chuyển thành tập cảnh báo tình chính $\mathcal{P}_{\text{cảnh báo}} = p \in \mathcal{P}_{\text{candidate}} \mid \mathcal{J}(p) = 1$, nâng cao đáng kể độ chính xác của việc phát hiện.

V. THỰC HIỆN

Hệ thống nguyên mẫu Liva được triển khai với khoảng 7000 dòng mã Python và 2000 dòng mã Java. Nó tích hợp IDA Pro 9.0 và Ghidra 10.4 để thực hiện dịch ngược và phân tích chức năng trên các tệp nhị phân của chương trình cơ sở và hiện hỗ trợ nhiều kiến trúc, bao gồm x86, ARM, MIPS và PowerPC. Để nhận dạng nguồn, Liva tận dụng các tính năng lập chỉ mục và phân tích chuỗi để suy ra các hàm liên quan đến đầu vào của người dùng HTTP và đánh dấu chúng là nguồn. Để nhận dạng phân chìm, hệ thống xử lý trước các tệp nhị phân để trích xuất các hàm thư viện chia sẻ phụ thuộc của chúng, áp dụng so khớp dựa trên quy tắc để xác định các hàm nguy hiểm tiềm ẩn và cải tiến thêm việc phân loại của chúng thông qua phân tích tác nhân thông minh.

Về cốt lõi, Liva kết hợp hệ thống đa tác nhân dựa trên LLM để hỗ trợ phân tích luồng dữ liệu liên thủ tục và suy luận về lỗ hổng. Tác nhân phân tích vết bản được xây dựng trên mô hình Qwen3-32B đã tinh chỉnh, được huấn luyện với 3.000 mẫu chức năng được trích xuất từ chương trình cơ sở trong thế giới thực để nắm bắt các mẫu lan truyền vết bản; kết quả của nó được tổ chức trong cơ sở dữ liệu đồ thị để lập mô hình tự động các đường dẫn luồng dữ liệu đa chức năng. Tác nhân suy luận lỗ hổng sử dụng LLM thương mại làm công cụ suy luận, tận dụng các kết quả lan truyền vết bản, biểu đồ cuộc gọi và mã dịch ngược để tự động suy ra các lỗ hổng tiềm ẩn và phân loại các loại của chúng (ví dụ: tràn bộ đệm, chèn lệnh). Bằng cách kết hợp phân tích tĩnh với suy luận đa tác nhân do LLM điều khiển, Liva đạt được khả năng phát hiện lỗ hổng hiệu quả và tự động trên các phần sụn không đồng nhất.

VI. ĐÁNH GIÁ

Trong phần này, chúng tôi tiến hành đánh giá Liva để trả lời các câu hỏi nghiên cứu sau:

RQ1: Liva xác định nguồn và mức chìm trong mã mục tiêu hiệu quả như thế nào?

RQ2: Thuật toán phân tích dòng hàm từ trên xuống của Liva hiệu quả như thế nào, dựa trên phân cụm nguồn và thuật toán hợp nhất chuỗi cuộc gọi của nó?

RQ3: Các mô hình ngôn ngữ lớn nguồn mở được chọn và tinh chỉnh trong Liva như thế nào? Làm thế nào để họ so sánh với các mô hình thương mại trong phân tích vết bản?

RQ4: Hiệu suất của Liva so với các công cụ phân tích vết bản tĩnh hiện có như thế nào?

RQ5: Liva có hiệu quả như thế nào trong việc phát hiện các lỗ hổng 0 ngày trong phần mềm IoT trong thế giới thực?

A. Thiết lập thử nghiệm

Cấu hình: Liva đã được triển khai trên Ubuntu 22.04 LTS với CPU Intel Core i9-12900KF 3,20 GHz (16 lõi, 24 luồng) và RAM DDR4 16 GB. Đối với các tác vụ tinh chỉnh mô hình quy mô lớn, chúng tôi đã sử dụng hai GPU NVIDIA RTX PRO 6000 Blackwell Server Edition, mỗi GPU được trang bị 96 GB bộ nhớ GDDR7 cực nhanh, cung cấp tổng cộng 192 GB

TABLE I
PERFORMANCE OF LLMs ON SOURCE/SINK IDENTIFICATION (COST UNIT: USD
PER 1 M TOKENS)

Model	Cost	Source				Sink			
		TP	FP	Prec	Rec	TP	FP	Prec	Rec
DeepSeek-v3	\$0.14	10	0	100%	100%	10	1	91%	100%
GPT-4o	\$2.50	10	0	100%	100%	10	1	91%	100%
Qwen-Max	\$0.33	10	0	100%	100%	10	5	67%	100%
Claude-Max	\$3	10	0	100%	100%	10	7	59%	100%

Each task uses 10 positive and 10 negative samples.

Bộ nhớ GPU để xử lý các nhu cầu tính toán. Môi trường phát triển bao gồm Python 3.8, PyTorch 1.12.0 với hỗ trợ đào tạo phân tán và CUDA 12.8.

Bộ dữ liệu chương trình cơ sở: Để đánh giá Liva, chúng tôi đã xây dựng một bộ dữ liệu chương trình cơ sở đa nguồn. Các mẫu chủ yếu được thu thập từ các bộ dữ liệu nghiên cứu có sẵn công khai, bao gồm SATC, KARONTE và FIRMAE, cũng như từ các bản phát hành chính thức của nhà cung cấp. Như được tóm tắt trong Bảng VII, bộ dữ liệu bao gồm 64 mẫu chương trình cơ sở từ 11 nhà cung cấp, bao gồm nhiều loại thiết bị như bộ định tuyến, điểm truy cập, bộ chuyển mạch, cổng VPN, bộ khuếch đại tín hiệu và hệ thống lưu trữ gần mạng. Tất cả các mẫu đều thuộc các dịch vụ web và thể hiện sự đa dạng trong kiến trúc hệ thống, cơ chế triển khai và chiến lược định tuyến, do đó phản ánh tính không đồng nhất trong thiết kế chương trình cơ sở và các biện pháp bảo mật giữa các nhà cung cấp khác nhau.

Tinh chỉnh tập dữ liệu: Để tinh chỉnh hiệu quả mô hình phân tích phần mềm điều khiển theo hướng dễ bị tổn thương, chúng tôi đã xây dựng một cách có hệ thống một tập dữ liệu chuyên dụng lấy từ các mẫu phần mềm cơ sở trong thế giới thực có chứa các lỗ hổng N-ngày đã biết. Cụ thể, trước tiên, chúng tôi xác định và trích xuất các chuỗi cuộc gọi dễ bị tấn công từ bộ sưu tập chương trình cơ sở và thu thập các mối quan hệ truyền tham số liên chức năng của chúng dưới dạng các mẫu dương tính. Để đảm bảo sự cân bằng trong quá trình đào tạo, chúng tôi đã chọn thêm các hàm không dễ bị tổn thương từ cùng một cơ sở mã chương trình cơ sở và trích xuất các mối quan hệ truyền tham số của chúng dưới dạng mẫu âm tính. Thông qua quy trình này, chúng tôi đã xây dựng tổng cộng 3.000 mẫu tinh chỉnh, bao gồm các loại chức năng và bối cảnh thực thi đa dạng, từ đó nâng cao khả năng thích ứng của mô hình với các tình huống không đồng nhất.

Ngoài ra, dựa trên bốn chỉ số lượng ảnh hưởng tham số được mô tả trong Phần IV-D1b, chúng tôi đã xây dựng 400 mẫu thử nghiệm để đánh giá một cách có hệ thống mức độ tổng quát và độ tin cậy của mô hình trong các kịch bản chuyển đổi tham số khác nhau.

Đường cơ sở: Chúng tôi đã chọn SATC [10] và Karonte [38] để so sánh làm đường cơ sở. Những kỹ thuật này đại diện cho công nghệ tiên tiến nhất trong phân tích lỗ hổng phần mềm. SATC sử dụng các nguồn được trích xuất từ giao diện người dùng với các hàm chìm được xác định trước để phân tích vết bản. Karonte tập trung vào việc xác định các lỗ hổng gây ra bởi sự tương tác giữa nhiều tệp nhị phân trong hình ảnh chương trình cơ sở. Chúng tôi không đưa vào các công cụ phân tích tĩnh khác do chúng phụ thuộc vào đặc tả quy tắc thủ công hoặc khả năng áp dụng hạn chế đối với các kịch bản phân tích chương trình cơ sở.

Số liệu: Để đánh giá tính hiệu quả của Liva trong việc phát hiện lỗ hổng phần mềm cơ sở, chúng tôi áp dụng 5 số liệu phân loại phổ biến: *Cảnh báo*, *Kết quả tích cực thực sự (TP)*, *Tiêu cực sai (FN)*, *Độ chính xác* và *Thu hồi*. *Cảnh báo* biểu thị tổng số lỗ hổng

cảnh báo do công cụ tạo ra cho một mẫu chương trình cơ sở nhất định; *TP* biểu thị số lượng lỗ hổng thực sự được công cụ xác định chính xác; *FN* đề cập đến số lượng lỗ hổng thực sự mà công cụ không phát hiện được. Dựa trên các giá trị này, chúng tôi tính toán thêm *Độ chính xác* và *Thu hồi*, được định nghĩa là $\text{Chính xác} = \frac{TP}{Alerts}$ và $\text{Thu hồi} = \frac{TP}{TP+FN}$.

Độ chính xác đo lường tỷ lệ lỗ hổng thực sự trong số tất cả các cảnh báo được báo cáo, phản ánh độ chính xác của kết quả phát hiện. Việc thu hồi đo lường tỷ lệ các lỗ hổng thực sự được phát hiện thành công, phản ánh mức độ hoàn chỉnh của việc phát hiện. Ngoài ra, chúng tôi cũng ghi lại thời gian chạy (*Thời gian*) của từng công cụ trên các mẫu phần mềm khác nhau, nhằm đánh giá hiệu quả của việc kiểm tra lỗ hổng.

B. RQ1: Nhận dạng nguồn và đích

1) Lựa chọn LLM để nhận dạng: Để chọn LLM thích hợp cho quy trình nhận dạng nguồn và điểm chìm của chúng tôi, trước tiên, chúng tôi đã tiến hành một nghiên cứu so sánh nhỏ bằng cách sử dụng bốn mô hình đại diện: DeepSeek-V3, GPT-4o, Qwen-Max và Claude-Max. Chúng tôi đã thu thập các đoạn chức năng từ phần mềm Tenda, D-Link và Linksys và chuẩn bị các bộ dữ liệu cân bằng cho cả hai tác vụ (mỗi tác vụ 10 mẫu dương tính và 10 mẫu âm tính). Tất cả các mô hình đều được truy vấn với cùng một mẫu lời nhắc, có trong phụ lục A và B, có sẵn trực tuyến.

Các kết quả được thể hiện trong Bảng I. Mặc dù cả bốn mô hình đều thực hiện tốt như nhau trong việc nhận dạng nguồn, nhưng hành vi của chúng khác nhau đáng kể trong nhiệm vụ chìm. DeepSeek-V3 và GPT-4o đạt được độ chính xác cao nhất, mỗi loại chỉ tạo ra một kết quả dương tính giả duy nhất, trong khi Qwen-Max và Claude-Max tạo ra nhiều hơn đáng kể. Vì DeepSeek-V3 mang lại độ chính xác tương đương với GPT-4o nhưng với chi phí suy luận thấp hơn nhiều, nên nó mang lại sự cân bằng thiết thực nhất cho cài đặt của chúng tôi. Vì lý do này, chúng tôi sử dụng DeepSeek-V3 làm LLM để nhận dạng nguồn và đích trong phần còn lại của bài viết.

2) Nhận dạng nguồn: Bảng II trình bày so sánh số lượng nguồn và phần chìm được Liva và SaTC xác định trên 15 mẫu chương trình cơ sở IoT.

Liva đã xác định được tổng cộng 6.723 nguồn, tăng 8,1% so với 6.217 được SaTC phát hiện. Trong hầu hết các trường hợp, Liva hoạt động tốt hơn SaTC, với mức cải thiện lớn nhất được quan sát thấy ở DrayTek Vigor3900 (207,9%). Tuy nhiên, đối với một số mẫu phần mềm cơ sở như Linksys WRT54 G, Tenda W15E và Trendnet TEW755AP, Liva báo cáo có ít nguồn hơn. Kiểm tra thủ công cho thấy sự sụt giảm này là do có nhiều kết quả dương tính giả trong SaTC, khiến kết quả của nó bị thổi phồng. Điều này cho thấy Liva đạt được độ chính xác cao hơn bằng cách giảm khả năng phát hiện giả, dẫn đến nhận dạng nguồn đáng tin cậy hơn về tổng thể.

3) Nhận dạng phần chìm: Để nhận dạng phần chìm, Liva đã phát hiện 142 chức năng so với 115 của SaTC, tăng 23,5%. Cải tiến đáng chú ý nhất xảy ra ở phần mềm Trendnet TEW824DRU, với mức tăng 75,0%. Những cải tiến này xuất phát từ khả năng của Liva trong việc xử lý các chức năng bao bọc trong chuỗi cuộc gọi sâu, nơi SaTC thường không truyền được dữ liệu. Trong một số chương trình cơ sở (ví dụ: TP-Link WPA8630, WavLink WN579A3, TOTOLink A3002R, Tenda W15E và Trendnet TEW755AP), cả hai công cụ đều tạo ra kết quả giống nhau, gợi ý lệnh gọi đơn giản hơn

BẢNG II SO SÁNH NHẬN DẠNG NGUỒN VÀ Sink
GIỮA LIVA và SATC TRÊN THIẾT BỊ IOT ĐẠI DIỆN

Vendor	Device	Program	Type	Arch	Source			Sink		
					SaTC	Liva	Increase %	SaTC	Liva	Increase %
Cisco	RV110W	httpd	VPN	MIPS	225	422	87.6	7	9	28.6
D-Link	DI8300	jhttpd	Router	MIPS	371	526	41.8	8	11	37.5
D-Link	DNS320L	account_mgr.cgi	NAS	ARM	52	90	73.1	7	10	42.9
NetGear	R8500	httpd	Router	ARM	1263	1307	3.5	9	11	22.2
NetGear	R7900	htpd	Router	ARM	1135	1250	10.1	8	10	25.0
TP-Link	WPA8630	httpd	Powerline Kit	MIPS	142	146	2.8	8	8	-
WavLink	WN579A3	login.cgi	WiFi Extender	MIPS	14	13	-7.1	7	7	-
Belkin	N600	webs	WiFi Extender	MIPS	682	750	10.0	7	10	42.9
TOTOLink	A3002R	boa	WiFi Extender	MIPS	657	785	19.5	7	7	-
Linksys	WRT54G	httpd	Router	MIPS	331	175	-47.1	7	8	14.3
Trendnet	TEW824DRU	ssi	AP	MIPS	130	171	31.5	8	14	75.0
Trendnet	TEW755AP	ssi	AP	MIPS	58	38	-34.5	7	7	-
Tenda	AX1806	tdhttpd	Router	ARM	244	286	17.2	8	11	37.5
Tenda	W15E	httpd	Router	ARM	824	490	-40.5	7	7	-
DrayTek	Vigor3900	mainfunction.cgi	VPN	ARM	89	274	207.9	10	12	20.0
Total	-	-	-	-	6217	6723	8.1	115	142	23.5

các cấu trúc. Nhìn chung, Liva cung cấp phạm vi bao phủ rộng hơn và chính xác hơn, nâng cao tính hoàn chỉnh của phân tích lỗ hổng.

4) *Phân tích FP và FN*: Để nhận dạng *nguồn*, LLM định vị một cách nhất quán và chính xác các chức năng liên quan đến đầu vào của người dùng, không có kết quả âm tính giả hoặc dương tính giả đáng kể nào được quan sát thấy trong các thử nghiệm. Điều này chủ yếu là do đặc điểm riêng biệt của các chức năng đó: một mặt, tên hàm thường phản ánh trực tiếp mục đích thu thập đầu vào bên ngoài (ví dụ: `websGetVar`, `get_cgi`, `cgiGetValue`); mặt khác, các mẫu đề trích xuất đầu vào của người dùng từ cấu trúc giao thức HTTP thường được cố định, giúp mô hình nắm bắt các mẫu này dễ dàng hơn. Tuy nhiên, kết quả dương tính giả nổi bật hơn đối với *phần chìm* được gói gọn trong các thư viện dùng chung. Các hàm này thường liên quan đến nhiều lớp lệnh gọi hàm, trong đó các tham số có thể được định dạng hoặc chuyển đổi thành các loại không phải chuỗi trong quá trình truyền. Ví dụ: trong hàm `send_msg_to_netctrl` của thiết bị Tenda, tham số `a1` được chuyển đến hàm `system` cuối cùng bằng cách sử dụng định dạng "%d", điều này ngăn cản việc giữ lại ngữ nghĩa chuỗi gốc, khiến LLM khó xác định liệu nó có phải là điểm tiêm lệnh thực hay không, dẫn đến kết quả dương tính giả.

C. RQ2: Đánh giá thuật toán phân tích luồng chức năng

Để giảm thiểu tính toán dư thừa trong phân tích vết bản, chiến lược tổng hợp hàm đã được sử dụng. Việc đánh giá được thực hiện theo hai khía cạnh: tổng hợp nguồn và tổng hợp đường dẫn nhảy cảm, với kết quả chi tiết được tóm tắt trong Bảng III.

1) *Tổng hợp nguồn*: Trong 15 mẫu chương trình cơ sở của bộ định tuyến, tổng cộng 6.723 điểm nguồn đã được xác định. Sau khi áp dụng tổng hợp, con số này giảm xuống còn 1.281, tương ứng với mức giảm tổng thể là 80,95%. Hầu hết các thiết bị đều đạt được tỷ lệ tổng hợp từ 70% đến 90%. Ví dụ: NetGear R8500 và TOTOLink A3002R lần lượt đạt 90,21% và 91,34%, trong khi ngay cả một thiết bị có ít nguồn hơn, WavLink

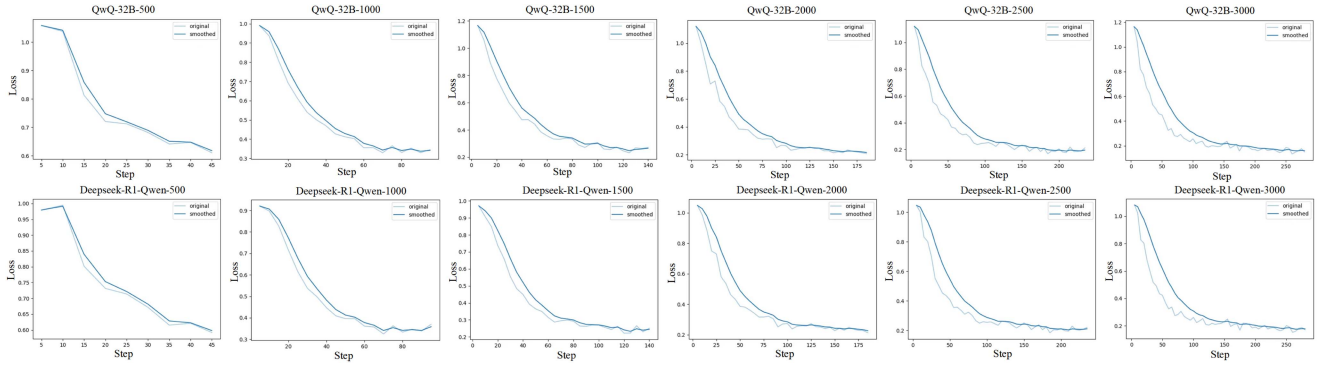
BẢNG III TỔNG HỢP ĐIỂM NGUỒN
VÀ KẾT QUẢ Sáp nhập CHỨC NĂNG NHẢY CẢM

Vendor Device	Source Point	Merged	Analysis Count %	Sensitive Function	Merged	Analysis Count %	FN Count
Cisco RV110W	422	104	75.36	468	90	80.77	0
D-Link DI8300	526	84	84.03	426	293	31.22	1
D-Link DNS320L	90	32	64.44	213	85	60.09	0
NetGear R8500	1307	128	90.21	720	272	62.22	0
NetGear R7900	1250	127	89.84	753	287	61.89	0
TP-Link WPA8630	146	44	69.86	321	132	58.88	1
WavLink WN579A3	13	2	84.62	13	6	53.85	0
Belkin N600	750	88	88.27	706	123	82.58	0
TOTOLink A3002R	785	68	91.34	415	111	73.25	0
Linksys WRT54G	175	29	83.43	68	19	72.06	0
Trendnet TEW824DRU	171	55	67.84	104	63	39.42	0
Trendnet TEW755AP	38	22	42.11	74	45	39.19	0
Tenda AX1806	286	50	82.52	320	151	52.81	1
Tenda W15E	490	140	71.43	719	310	56.88	0
DrayTek Vigor3900	274	108	60.58	288	155	46.18	0
Total	6,723	1,281	80.95	5,534	2,132	61.48	

WN579A3, đạt 84,62%. Những kết quả này chứng minh rằng phương pháp đề xuất loại bỏ một cách hiệu quả các nguồn dư thừa và giảm đáng kể quy mô phân tích.

2) *Tổng hợp đường dẫn nhảy cảm*: Đối với các đường dẫn chức năng nhảy cảm, ban đầu đã phát hiện 5.534 đường dẫn ứng cử viên. Sau khi tổng hợp, con số này giảm xuống còn 2.132, mang lại mức giảm chung là 61,48%. Tỷ lệ giảm khác nhau giữa các thiết bị. Belkin N600 có mức giảm cao 82,58%, trong khi D-Link DI8300 chỉ đạt 31,22%. Điều này chỉ ra rằng một số mẫu chương trình cơ sở chứa một số lượng lớn các đường dẫn chồng chéo có thể được hợp nhất, trong khi các mẫu khác có nhiều đường dẫn phân tán hơn với khả năng tập hợp thấp hơn. Nhìn chung, phương pháp này làm giảm đáng kể việc dò đường dư thừa và nâng cao hiệu quả phân tích vết bản.

Tập hợp đường dẫn nhảy cảm được tiến hành trong giai đoạn phân tích vết bản để hợp nhất nhiều đường dẫn nhảy cảm trong cùng một chức năng gọi để phân tích chung. So với phân tích đường dẫn riêng lẻ, số lượng yêu cầu phân tích giảm đi đáng kể sau khi tổng hợp. Tuy nhiên, có một mức độ âm tính giả nhất định được quan sát thấy trên một số thiết bị, bao gồm D-Link, TP-Link và Tenda. Sau khi kiểm tra kỹ hơn, chúng tôi thấy rằng, như trong Hình 12 (Phụ lục D, có sẵn trực tuyến), mẫu âm tính giả chứa hai trang web cuộc gọi rất giống nhau trong



Hình 7. Tình hình đường cong suy hao của QwQ-32B và DeepSeek-R1-Qwen theo các thang dữ liệu khác nhau.

chức năng tương tự. Sau khi tổng hợp, hai địa điểm gọi này được hợp nhất thành một phiên bản phân tích duy nhất, điều này có thể ảnh hưởng đến độ chính xác của cảnh báo được báo cáo.

D. RQ3: Thử nghiệm tinh chỉnh LLM

Chúng tôi đã chọn bốn mô hình lớn ngữ nguyên mở có khả năng phân tích mã nguồn làm mô hình cơ sở để tinh chỉnh: DeepSeek-R1-Distill-Qwen-32B [38], DeepSeek-Coder-33B-Instruct [39], [40], QwQ-32B [41] và Qwen3-32B [42]. Trong các thí nghiệm sau, chúng tôi sẽ đánh giá chúng để xác định mô hình phù hợp nhất làm cốt lõi của tác nhân Liva.

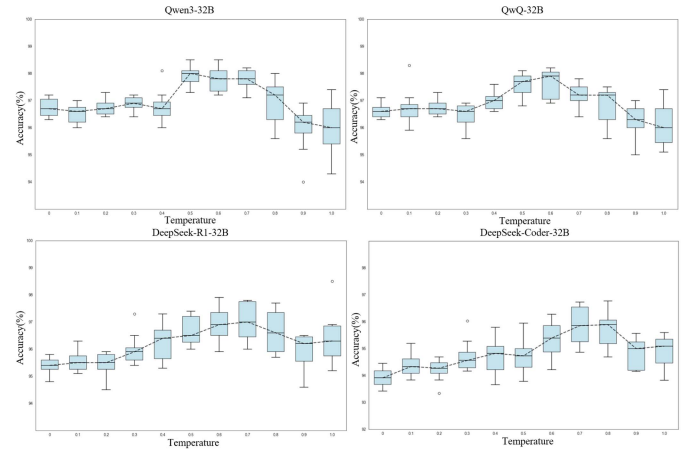
1) *Xác định quy mô dữ liệu tinh chỉnh*: Để xác định quy mô dữ liệu phù hợp cho việc tinh chỉnh, chúng tôi đã thực hiện các thử nghiệm trên QwQ-32B và DeepSeek-R1-Qwen với dữ liệu huấn luyện từ 500 đến 3000 mẫu, tăng thêm 500 mẫu mỗi bước. Kết quả trong Hình 7 cho thấy quy mô dữ liệu có tác động trực tiếp đến sự hội tụ của mô hình. Với dữ liệu nhỏ (500-1000 mẫu), độ mất mát cao và đường cong huấn luyện không ổn định. Khi kích thước dữ liệu tăng lên, độ hội tụ trở nên nhanh hơn, đường cong huấn luyện trở nên mượt mà hơn và độ mất mát cuối cùng sẽ giảm.

Đối với QwQ-32B, Bảng IX (Phụ lục C, có sẵn trực tuyến) cho thấy tổn thất là 1,1635 ở bước 8, giảm nhanh xuống 0,4503 ở bước 52 và đạt giá trị thấp nhất là 0,1330 ở bước 262. Khi thang dữ liệu vượt quá 2000, tổn thất tiếp tục giảm nhưng mức độ cải thiện giảm đi. Ở 3000 mẫu, độ suy hao thấp nhất và đường cong ổn định nhất, thể hiện độ hội tụ tốt nhất. DeepSeek-R1-Qwen cho thấy một mô hình tương tự, trong đó dữ liệu lớn hơn sẽ cải thiện cả tính ổn định và khả năng hội tụ.

Do đó, khi xem xét tốc độ hội tụ, tổn thất cuối cùng và độ ổn định huấn luyện, chúng tôi đã chọn 3000 mẫu làm thang đo tinh chỉnh mặc định cho các thử nghiệm sau.

2) Lựa chọn mô hình nguồn mở và cài đặt tham số:

a) *Tác động của nhiệt độ đến độ chính xác*: Thông số nhiệt độ đóng vai trò quan trọng trong việc điều chỉnh tính sáng tạo và đa dạng của nội dung do các mô hình ngôn ngữ lớn tạo ra. Để đánh giá một cách có hệ thống tác động của nhiệt độ đến độ chính xác trong phân tích của Liva, chúng tôi đã xây dựng một tập dữ liệu chứa 400 mẫu thử nghiệm bao gồm bốn tình huống truyền tham số điển hình được mô tả trong Phần IV-D. Các thử nghiệm được tiến hành trên bốn mô hình nguồn mở đã được tinh chỉnh, với các giá trị nhiệt độ được đặt từ 0,0 đến 1,0 với mức tăng 0,1 trên 11 độ dốc. Mỗi



Hình 8. Tác động của nhiệt độ đến độ chính xác phân tích của các mô hình nguồn mở đã được tinh chỉnh.

cấu hình thử nghiệm được thực hiện trong 10 vòng để đảm bảo độ tin cậy thống kê của kết quả.

Hình 8 minh họa mối quan hệ giữa các thông số nhiệt độ và độ chính xác của phân tích mô hình. Trong phạm vi nhiệt độ thấp (0,0-0,4), độ chính xác của mô hình vẫn tương đối thấp với độ biến thiên hạn chế, có thể là do các mẫu đầu ra quá xác định đã hạn chế khả năng phân tích cú pháp cho các đường truyền tham số phức tạp. Khi nhiệt độ tăng lên phạm vi vừa phải (0,5-0,7), độ chính xác được cải thiện đáng kể và đạt các giá trị đỉnh tương ứng, cho thấy rằng các mô hình đã đạt được khả năng khám phá nâng cao trong khi vẫn duy trì độ ổn định suy luận. Khi nhiệt độ tiếp tục tiến vào phạm vi nhiệt độ cao (0,8-1,0), độ chính xác bắt đầu có những biến động đáng chú ý, cho thấy rằng tính ngẫu nhiên quá mức sẽ làm tổn hại đến tính ổn định của phân tích mã.

Nhìn chung, nhiệt độ 0,5-0,7 mang lại sự cân bằng tối ưu giữa độ ổn định và khả năng phát hiện cho Liva.

b) *Tác động của các mô hình đến hiệu quả phân tích*: Hình 8 báo cáo độ chính xác của bốn mô hình nguồn mở, trong đó Qwen3-32B và QwQ-32B đạt độ chính xác trung bình cao nhất lần lượt là 97,79% và 97,91%. Bảng V (bảng trên) so sánh thêm thời gian phân tích của họ trên bốn loại dữ liệu thử nghiệm. Kết quả cho thấy Qwen3-32B luôn được yêu cầu

BẢNG IV TIÊU THỤ TOKEN VÀ CHI PHÍ PHÂN TÍCH DLINK DIR816

Model	Token Price (USD/1M)	Tokens Used	Cost (USD)
GPT-4o	\$2.50/\$10.00	632,880/2,782	\$1.61
Gemini-2.5-Pro	\$2.50/\$15.00	569,592/3,108	\$1.47
Qwen-Max	\$0.33/\$6.20	537,948/3,218	\$0.20
Claude-3.7-Sonnet	\$3.00/\$15.00	696,168/2,983	\$2.13
Liva	-/-	137,326/2,636	-

BẢNG V SO SÁNH HIỆU SUẤT CÁC PHƯƠNG PHÁP PHÁT HIỆN KHÁC NHAU

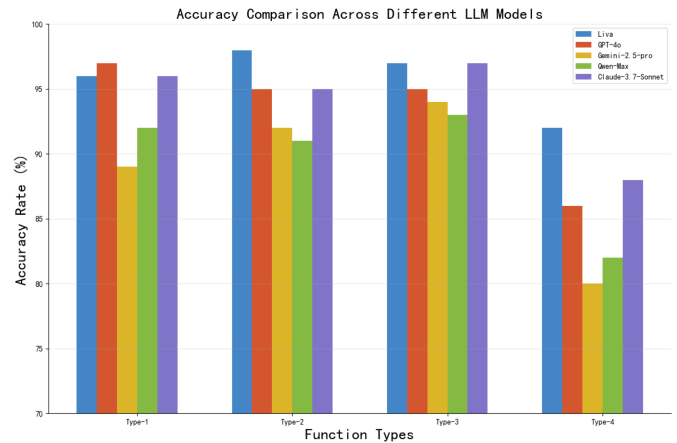
Model	Type I Time	Type II Time	Type III Time	Type IV Time
Qwen3-32B	13:20	12:45	14:27	18:16
QwQ-32B	20:35 (↑7:15)	18:17 (↑5:32)	19:21 (↑4:54)	25:29 (↑7:13)
DeepSeek-R1-32B	18:41 (↑5:21)	17:51 (↑5:06)	18:25 (↑3:58)	24:48 (↑6:32)
DeepSeekCoder-33B	17:36 (↑4:16)	17:12 (↑4:27)	18:43 (↑4:16)	22:39 (↑4:23)
GPT-4o	14:56 (↑1:36)	15:04 (↑2:19)	16:48 (↑2:21)	17:41 (↓0:35)
Gemini-2.5-pro	16:28 (↑3:08)	15:11 (↑2:26)	17:26 (↑2:59)	21:53 (↑3:37)
Qwen-Max	15:43 (↑2:23)	13:37 (↑0:52)	12:38 (↓1:49)	19:34 (↑1:18)
Claude-3.7-Sonnet	14:17 (↑0:57)	12:39 (↓0:06)	16:12 (↑1:45)	17:36 (↓0:40)

thời gian xử lý ngắn nhất. Ngược lại, mặc dù QwQ-32B đạt mức độ chính xác tương đương nhưng thời gian phân tích của nó dài hơn đáng kể, với mức tăng tối đa khoảng bảy phút so với Qwen3-32B. Hai mẫu DeepSeek, DeepSeek-R1-32B và DeepSeek-Coder-33B, thể hiện hiệu suất ở mức trung bình, nhưng thời gian xử lý của chúng thường dài hơn 4-6 phút so với Qwen3-32B trong tất cả các tác vụ.

Nhìn chung, Qwen3-32B mang lại sự cân bằng thuận lợi nhất giữa độ chính xác và hiệu quả. QwQ-32B đạt được độ chính xác cao nhưng chi phí tính toán cao hơn, trong khi hai mô hình DeepSeek cho kết quả vừa phải và không vượt qua được Qwen3-32B ở cả hai khía cạnh. Dựa trên những phát hiện này, Qwen3-32B đã được chọn làm mô hình cơ sở để tính chính.

3) So sánh mô hình thương mại: Để đánh giá toàn diện hiệu suất của mô hình Liva được đề xuất (tính chính từ Qwen3-32B), chúng tôi đã chọn bốn mô hình thương mại chính thống làm đường cơ sở, đó là GPT-4o [43], Gemini-2.5-pro [44], Qwen-Max [45] và Claude-3.7-Sonnet [46]. Các thí nghiệm so sánh được thực hiện từ hai góc độ: độ chính xác và thời gian phân tích.

Về độ chính xác, Liva đạt được hiệu suất hàng đầu trên các bộ dữ liệu Loại 2, Loại 3 và Loại 4, với độ chính xác lần lượt là 98,21%, 97,33% và 92,58% (Hình 9). Nó chỉ thấp hơn một điểm phần trăm so với GPT-4o trên Loại 1, trong khi ngang bằng hoặc vượt qua Claude-3.7-Sonnet trong các trường hợp khác, do đó chứng tỏ độ chính xác tổng thể luôn cao. Về hiệu quả phân tích, một số mô hình thương mại nhất định thể hiện tốc độ xử lý nhanh hơn trong các tác vụ cụ thể, nhưng điều này thường làm giảm độ chính xác. Ví dụ: trên tập dữ liệu Loại 3, Qwen-Max đã giảm thời gian chạy xuống 1 phút 49 giây so với Liva, nhưng độ chính xác của nó chỉ là 93%, thấp hơn 4 điểm phần trăm so với Liva. Tương tự, GPT-4o nhanh hơn Liva 35 giây trên tập dữ liệu Loại 4, tuy nhiên độ chính xác của nó giảm xuống 86%, thấp hơn 6 điểm phần trăm.



Hình 9. So sánh độ chính xác của phân tích giữa Liva và các mô hình thương mại lớn trên bốn bộ dữ liệu ảnh xạ tham số.

Ngoài độ chính xác và thời gian chạy, chúng tôi còn kiểm tra thêm mức tiêu thụ tài nguyên liên quan đến việc tiến hành phân tích vết bản bằng các mô hình khác nhau. Liva phân sụn D-Link DIR816 làm ví dụ, LLM thương mại phát sinh việc sử dụng mã thông báo trong phạm vi 500k-600k mã thông báo trên mỗi nhị phân. Ngược lại, Liva cho thấy mức tiêu thụ token giảm đáng kể. Mức giảm này xuất phát từ thực tế là LLM thương mại thường yêu cầu những lời nhắc dài để chỉ định rõ ràng các yêu cầu nhiệm vụ trong quá trình tương tác dựa trên API, trong khi mô hình được tinh chỉnh như Liva có thể diễn giải trực tiếp các đầu vào phân tích có cấu trúc với hướng dẫn theo ngữ cảnh tối thiểu. Mặc dù chi phí mỗi chương trình của các mô hình thương mại vẫn tương đối thấp và giá token của Liva không thể được định lượng đầy đủ do nó được triển khai cục bộ, nhưng một chương trình cơ sở thường chứa nhiều tệp nhị phân cần phân tích. Do đó, khi mở rộng quy mô sang các tập dữ liệu phân sụn lớn, Liva mang lại lợi thế đáng kể về việc giảm mức sử dụng mã thông báo và chi phí vận hành tổng thể.

Nhìn chung, mặc dù Liva yêu cầu thời gian phân tích lâu hơn một chút so với một số mẫu thương mại cho một số nhiệm vụ nhất định nhưng nó luôn duy trì độ chính xác cao hơn, qua đó chứng tỏ hiệu quả thực tế vượt trội trong việc hỗ trợ phân tích vết bản.

4) Tác động của Chất lượng dịch ngược đến độ chính xác của phân tích: Vì việc phân tích phụ thuộc rất nhiều vào độ chính xác của mã dịch ngược nên lỗi dịch ngược có thể ảnh hưởng đến độ chính xác của kết quả phân tích vết bản. Để đánh giá mức độ chắc chắn của các lỗi như vậy, tác động do các công cụ giải mã khác nhau gây ra sẽ được nghiên cứu sâu hơn. So với IDA, Ghidra dễ mắc lỗi dịch ngược hơn trong quá trình khôi phục luồng điều khiển, suy luận kiểu và con trỏ cũng như tái cấu trúc biến, điều này có thể làm biến dạng đáng kể cả đường truyền lan truyền vết bản trong và giữa các thủ tục. Đối với mỗi loại trong số ba loại này, 10 mẫu đại diện được chọn từ mã dịch ngược do Ghidra tạo ra và được phân tích cùng với các kết quả dịch ngược tương ứng do IDA tạo ra bằng cách sử dụng khung phân tích vết bản Liva. Kết quả so sánh được tóm tắt trong Bảng VI.

Kết quả chỉ ra rằng chất lượng mã dịch ngược của Ghidra thấp hơn so với IDA, dẫn đến giảm đáng kể độ chính xác của phân tích vết bản trên ba loại lỗi.

BẢNG VI SO SÁNH ĐỘ CHÍNH XÁC
CỦA CÁC BỘ PHÂN GIẢI KHÁC NHAU THEO BA LOẠI LỖI
ĐIỂN HÌNH

Decompiler	Control-Flow Recovery	Type & Pointer Inference	Variable Recovery
IDA	10	10	10
Ghidra	9	6	5

Lỗi khôi phục luồng điều khiển có tác động hạn chế, trong khi lỗi suy luận kiểu và tái cấu trúc biến có ảnh hưởng rõ rệt nhất, làm giảm độ chính xác xuống chỉ còn 50% -60%. Điều này cho thấy những thiếu sót về loại và khả năng phục hồi biến đổi trong Ghidra là những yếu tố chính ảnh hưởng đến độ chính xác của phân tích.

E. RQ4: So sánh với đường cơ sở

Bảng VII tóm tắt so sánh toàn diện giữa Liva và các hệ thống cơ sở SaTC và Karonte trên 65 mẫu chương trình cơ sở được thu thập từ 11 nhà cung cấp IoT lớn, bao gồm 370 lỗ hổng đã được xác nhận, bao gồm 64 lỗ hổng zero-day. Kết quả cho thấy Liva vượt trội so với đường cơ sở trên nhiều chỉ số.

1) *Hiệu quả phát hiện*: Trên tập dữ liệu đánh giá, Liva đạt tỷ lệ thu hồi trung bình là 98,1%, xác định thành công 363 trong số 370 lỗ hổng được xác nhận chỉ với 6 lỗi. Ngược lại, SaTC chỉ đạt mức thu hồi 14,6% (316 lần bỏ lỡ), trong khi Karonte chỉ đạt mức thu hồi 3,8% (356 lần bỏ lỡ), cả hai đều bị thiếu phạm vi phủ sóng nghiêm trọng. Hơn nữa, Liva duy trì hiệu suất ổn định giữa các nhà cung cấp khác nhau, cho thấy sự mạnh mẽ của nó trước các biến thể trong kiến trúc và triển khai chương trình cơ sở.

2) *Phân tích chính xác*: Trong khi duy trì mức thu hồi cao, Liva cũng đạt được độ chính xác 74,6%, với tổng số 487 cảnh báo, vượt trội đáng kể so với SaTC (40,3%, 134 cảnh báo) và Karonte (21,5%, 65 cảnh báo). Điều này cho thấy kiến trúc đa tác nhân của Liva không chỉ hiệu quả trong việc đạt được khả năng bao phủ toàn diện các lỗ hổng bảo mật—bao gồm một số lượng đáng kể các lỗ hổng zero-day—mà còn trong việc giảm các kết quả dương tính giả.

3) *Hiệu quả tính toán*: Liva hoàn thành toàn bộ phân tích trong 4.139 phút (68,9 giờ), trong khi SaTC và Karonte cần lần lượt là 27.161 phút (452,7 giờ) và 22.632 phút (377,2 giờ). Do đó, Liva đã đạt được khoảng $6.7 \times$ và $5.6 \times$ tốc độ so với hai đường cơ sở. Những kết quả này nhấn mạnh rằng Liva có khả năng giảm đáng kể chi phí phân tích trong khi vẫn duy trì độ chính xác phát hiện cao.

4) *Phân tích FP và FN cho Liva*: Trong phân tích kết quả dương tính giả, chúng tôi nhận thấy rằng Cisco (62,5%), Linksys (64,7%) và D-Link (69,4%) có độ chính xác tương đối thấp, chủ yếu là do ba lý do. Đầu tiên, một số chương trình cơ sở D-Link bao gồm chức năng lọc tiêu lệnh SpecSymbol2BackSlash, giúp thoát các ký tự đặc biệt. Mặc dù về mặt lý thuyết, bộ lọc có thể được bỏ qua bằng cách sử dụng %0a, nhưng ký tự này không thể được đưa vào khi hàm nguồn là cgiFormStringNoNewlines, dẫn đến các trường hợp bị gán cờ không chính xác là dễ bị tổn thương trong quá trình phân tích. Thứ hai, trong các thiết bị của Cisco và Linksys, một số chức năng nguồn nhất định thiếu đường dẫn định tuyến hợp lệ, khiến cho việc phân tích vết bản bị ảnh hưởng.

không thể phổ biến, nhưng các chức năng này vẫn bị xác định nhầm là dễ bị tấn công. Thứ ba, khi xử lý các tham số HTTP, Liva có thể kích hoạt định tuyến thứ cấp. Vì cảnh báo được đưa ra ở cấp nguồn nên điểm nguồn giống nhau có thể được sử dụng lại trên các đường dẫn định tuyến khác nhau, dẫn đến cảnh báo trùng lặp. Những yếu tố này trực tiếp gây ra tỷ lệ FP cao trong các thiết bị này.

Khi phân tích các kết quả âm tính giả, chúng tôi đã xem xét sáu trường hợp từ Netgear, TP-Link, Linksys và Tenda. Vấn đề chính nằm ở việc Liva không có khả năng nhận ra việc lưu trữ và truy xuất dữ liệu đa chức năng, điều này ngăn cản việc lập mô hình chính xác về việc lan truyền vết bản và dẫn đến việc phát hiện bị bỏ sót. Ngoài ra, trong CVE-2025-7094 của Belkin, hàm formBSSetSitesurvey tạo ra phần nội dung dịch ngược quá dài. Trong quá trình giải mã, các câu lệnh truyền dữ liệu chính bị bỏ qua, khiến cho quá trình phân tích vết bản không thành công.

5) *Phân tích FP và FN cho SaTC và Karonte*: Kết quả dương tính giả của SaTC chủ yếu bắt nguồn từ việc nhận dạng nguồn không chính xác trong các tệp nhị phân phụ trợ. Một số lượng đáng kể các nguồn được xác định không tương ứng với các đường định tuyến thực tế, do đó tạo ra các cảnh báo giả.

Ngược lại, những tiêu cực sai lầm của Karonte chủ yếu là do nó nhúng mạnh vào việc truy vấn dữ liệu giữa các nhà phân, trong khi bỏ qua đầu vào của người dùng bắt nguồn từ các yêu cầu HTTP. Do đó, một số nguồn đầu vào quan trọng vẫn không bị phát hiện.

Hơn nữa, cả hai công cụ đều có chung nguyên nhân gây ra kết quả âm tính giả. Đầu tiên, việc nhận dạng nguồn của chúng không đầy đủ, gây khó khăn cho việc nắm bắt các nguồn thiếu mã giao diện người dùng tương ứng. Thứ hai, cả hai công cụ đều gặp khó khăn trong việc nhận biết hiệu quả các chức năng bao bọc của các cuộc gọi nguy hiểm trong các thư viện được liên kết động, dẫn đến việc phát hiện bị bỏ sót.

F. RQ5: Phát hiện lỗ hổng 0 ngày

Như được hiển thị trong Bảng VIII, Liva đã phát hiện thành công 64 lỗ hổng chưa xác định trước đây (0 ngày) trên 15 mẫu thiết bị IoT khác nhau, hầu hết trong số đó liên quan đến việc chen lệnh (CI) và hỏng bộ nhớ. Những lỗ hổng này đã được báo cáo cho các nhà cung cấp tương ứng và được xác nhận, với 39 mã nhận dạng CVE hoặc CNVD chính thức đã được chỉ định. Kết quả chứng minh rằng Liva có thể phát hiện một cách hiệu quả các lỗ hổng trong chương trình cơ sở IoT trong thế giới thực và phát hiện các vấn đề bảo mật mà các phương pháp cơ bản không xác định được, làm nổi bật khả năng khái quát hóa mạnh mẽ trong các tình huống thực tế. Phân tích sâu hơn chỉ ra rằng những lý do chính dẫn đến việc bỏ sót các đường cơ sở có thể được tóm tắt như sau: 1) xác định không đầy đủ các nguồn hoặc bề chứa, và 2) không có khả năng nắm bắt được các mối quan hệ giữa các biến trong quá trình phân tích vết bản.

VII. CÔNG VIỆC LIÊN QUAN

A. Phân tích vết bản

Phân tích vết bản là một kỹ thuật phân tích bảo mật chương trình theo dõi luồng dữ liệu không đáng tin cậy từ điểm nguồn đến các điểm hoạt động nhạy cảm (bàn chìm) để phát hiện các lỗ hổng bảo mật tiềm ẩn [5]. Trong phân tích mã nguồn, FlowDroid [47] và TAJIS [48] đạt được khả năng theo dõi vết bản chính xác bằng cách xây dựng biểu đồ luồng điều khiển và biểu đồ luồng dữ liệu. Đối với các ứng dụng web, RIPS [49] và

BẢNG VII HIỆU SUẤT PHÁT HIỆN LỖ LỖ CỦA LIVA, SATC VÀ KARONTE TRÊN PHẦN MỀM IOT (ĐƠN VỊ THỜI GIAN: PHÚT)

Vendor	Device	Samples	Vuln	Liva						SaTC						Karonte					
				Alerts	TP	FN	Prec(%)	Recal(%)	Time	Alerts	TP	FN	Prec(%)	Recal(%)	Time	Alerts	TP	FN	Prec(%)	Recal(%)	Time
Cisco	RV	1	5(4)	8	5	0	62.5%	100.0%	35	7	1	4	14.3%	20.0%	197	4	1	4	25.0%	20.0%	133
D-Link	DIR/DNS/DCS/DI	10	59(33)	85	59	0	69.4%	100.0%	633	21	6	53	28.6%	10.7%	4,135	17	3	56	17.7%	5.1%	3,373
NetGear	WNR/RAC	14	12(2)	13	11	1	84.6%	91.7%	789	9	2	10	22.2%	16.7%	8,193	11	3	9	27.3%	25.0%	6,211
TP-Link	WR/WPA	5	10(2)	12	9	1	75.0%	90.0%	366	3	2	8	66.7%	20.0%	1,126	1	1	9	100.0%	10.0%	928
WavLink	WN	2	23(4)	27	23	0	85.2%	100.0%	46	8	1	22	25.0%	4.3%	566	4	0	23	0.0%	0.0%	781
BelKin	N	1	15(1)	16	14	1	87.5%	93.3%	68	7	3	12	42.9%	20.0%	256	7	2	13	28.6%	13.3%	389
Totolink	X/A/N/C/E/CS	10	82(2)	113	82	0	72.6%	100.0%	625	18	5	77	27.8%	6.1%	4,757	0	0	82	0.0%	0.0%	817
Linksys	WRT/RE/E	3	22(4)	34	22	0	64.7%	100.0%	221	0	0	22	0.0%	0.0%	173	3	0	22	0.0%	0.0%	693
Trendnet	TEW	4	27(4)	32	26	1	81.3%	96.3%	318	9	3	24	33.3%	11.1%	1,847	5	1	26	20.0%	3.7%	1,819
Tenda	AC/AX/FH/W	11	96(7)	126	93	3	73.8%	96.9%	737	49	31	65	46.2%	63.3%	4,946	8	3	93	37.5%	3.1%	5,760
DrayTek	V	3	19(1)	21	19	0	90.5%	100.0%	301	13	0	19	0.0%	0.0%	965	5	0	19	0.0%	0.0%	1,728
Total	-	64	370(64)	487	363	7	74.6%	98.1%	4,139	144	54	316	37.5%	14.6%	27,161	65	14	356	21.5%	3.8%	22,632

BẢNG VIII CÁC LỖ LỖ ZERO-DAY ĐƯỢC LIVA PHÁT HIỆN TRONG BỘ DỮ LIỆU PHẦN MỀM

Vendor & Model	Number	Unknown Vulnerabilities
Cisco RV110W	4	4 unassigned
D-Link DI8300	6	CNVD-2024-48723 to CNVD-2024-48725, CNVD-2024-49420 to CNVD-2024-49422
D-Link DNS320L	27	CVE-2024-7828 to CVE-2024-7832, CVE-2024-7715, CVE-2024-7849, CVE-2024-8127 to CVE-2024-8134, CVE-2024-8210 to CVE-2024-8214, CVE-2024-7922, CNVD-2024-47670 to CNVD-2024-47675
NetGear R8500	1	CNVD-2024-47646
NetGear R7900	1	1 unassigned
TP-Link WPA8630	2	2 unassigned
WavLink WN579A3	4	CNVD-2025-18059, CNVD-2025-18060, 2 unassigned
Belkin N600	1	1 unassigned
TOTOLINK A3002R	2	2 unassigned
Linksys WRT54G	4	CVE-2024-41281, CVE-2024-8408, 2 unassigned
Trendnet TEW824DRU/	1	1 unassigned
Trendnet TEW755AP	3	3 unassigned
Tenda AX1806	2	CNVD-2024-47661, 1 unassigned
Tenda W15E	5	5 unassigned
DrayTek Vigor3900	1	1 unassigned
Total	64	

Pixy [50] xây dựng các mô hình lan truyền vết bản để phát hiện các lỗ hổng như XSS và SQL SQL. Trong các ứng dụng di động, TaintDroid [51] triển khai tính năng theo dõi vết bản động ở cấp hệ thống Android, trong khi IccTA [52] tập trung vào phân tích lan truyền vết bản để liên lạc giữa các thành phần. Tuy nhiên, việc phân tích dấu vết cho các chương trình nhị phân của thiết bị IoT phải đối mặt với những thách thức lớn hơn. Các vấn đề chính bao gồm thiếu thông tin cấu trúc chương trình, xử lý bước nhảy gián tiếp phức tạp và tối ưu hóa mã tối ưu hóa [53]. Những vấn đề này thường yêu cầu các kỹ thuật thực thi biểu tượng để giải quyết [54].

B. Phân tích tĩnh trong phần sụn

Phần sụn IoT thường xuất hiện dưới dạng nhị phân nguồn đóng thiếu mã nguồn và thông tin gỡ lỗi. Các phương pháp kiểm tra mã và kiểm tra động truyền thống trở nên khó áp dụng. Ngược lại, phân tích nhị phân tĩnh hoạt động mà không cần môi trường thời gian chạy, điều này khiến nó trở nên quan trọng đối với bảo mật phần sụn. DTaint [8] triển khai theo dõi vết bản thông qua biểu đồ phụ thuộc dữ liệu ở cấp độ chức năng và sử dụng phân tích biểu đồ lệnh gọi từ dưới lên để giảm độ phức tạp. Tuy nhiên, phương pháp này cho thấy độ chính xác kém khi xử lý các luồng điều khiển phức tạp và các con trỏ hàm. Việc nhận dạng nguồn và đích dựa trên heuristic cung cấp phạm vi bao phủ hạn chế. SATC [13] xác định nguồn đầu vào bằng cách sử dụng các từ khóa được chia sẻ và định vị các điểm nguồn tiềm năng bằng cách phân tích các mẫu từ khóa trong giao diện truyền thông. Tuy nhiên, cách tiếp cận này chủ yếu dựa vào việc so khớp chuỗi và hoạt động kém trong các tình huống làm xáo trộn mã hoặc giao diện động. Karonte [12] tập trung vào các tương tác đa nhị phân và có thể mô hình hóa luồng dữ liệu cũng như luồng điều khiển để phát hiện các lỗ hổng giữa các thành phần. Tuy nhiên, phương pháp này vẫn còn thô trong phân tích lan truyền chỉ tiết trong các tệp nhị phân đơn lẻ.

HermeScan [55] giảm tính toán dư thừa thông qua các chiến lược lan truyền lũy tiến và cải thiện hiệu quả. Tuy nhiên, phương pháp này dựa trên các quy tắc được xác định trước và gặp khó khăn với các mẫu mã mới cũng như tối ưu hóa trình biên dịch.

C. Phân tích được hỗ trợ bởi LLM

Trong những năm gần đây, các nhà nghiên cứu đã tích cực khám phá việc kết hợp khả năng phân tích mã của các mô hình ngôn ngữ lớn với các kỹ thuật phân tích chương trình truyền thống để nâng cao hiệu quả kiểm tra mã. LLMDFA [56] đề xuất khung phân tích luồng dữ liệu được hỗ trợ bởi các mô hình ngôn ngữ lớn, khám phá các ý tưởng ứng dụng trong phân tích mã nguồn. LATTE [26] đề xuất phương pháp phân tích vết bản nhị phân tĩnh đầu tiên được điều khiển bởi các mô hình ngôn ngữ lớn mà không yêu cầu tùy chỉnh thủ công các quy tắc lan truyền vết bản. IRIS [57] đề xuất một phương pháp kết hợp thần kinh-ký hiệu tích hợp một cách có hệ thống các mô hình ngôn ngữ lớn với các kỹ thuật phân tích tĩnh để phát hiện các lỗ hổng bảo mật phức tạp cần phân tích toàn bộ kho lưu trữ. MFT-Coder [58] nâng cao khả năng của mô hình ngôn ngữ mã lớn thông qua các khung tinh chỉnh đa tác vụ, chứng minh rằng việc đào tạo chung có những lợi thế đáng kể so với việc tinh chỉnh độc lập. Vul-RAG [59] thiết kế một khung tạo thể hệ tăng cường truy xuất (RAG) ở cấp độ kiến thức [60] để cải thiện hơn nữa độ chính xác của việc nhận dạng lỗ hổng.

Để giải quyết các thách thức trong phân tích vết bản tĩnh nhị phân của phần sụn IoT, chúng tôi đề xuất nguyên mẫu Liva. Liva sử dụng các cơ chế cộng tác đa tác nhân mô hình lớn để nâng cao hiểu biết về chức năng và ngữ nghĩa của mã chương trình cơ sở. Hệ thống có thể tự động xác định các điểm nguồn và điểm chìm, đồng thời phân tích mối quan hệ truyền tham số giữa các chức năng, hỗ trợ phân tích vết bản và xác định lỗ hổng tự động.

VIII. PHÂN KẾT LUẬN

Bài viết này trình bày một giải pháp đa tác nhân LLM, Liva, được thiết kế để phân tích vết bản của các nhị phân chương trình cơ sở IoT. Cách tiếp cận này giải quyết ba hạn chế chính của máy phân tích vết bản truyền thống: (i) không đủ độ chính xác trong việc xác định nguồn và bồn chứa, thường dẫn đến kết quả dương tính giả và âm tính giả; (ii) tính toán dư thừa quá mức do các mối quan hệ gọi hàm phức tạp, dẫn đến hiệu quả thấp; và (iii) phụ thuộc nhiều vào kiến thức chuyên môn để xác định các quy tắc lan truyền vết bản, dẫn đến phạm vi bao phủ không đầy đủ. Liva tận dụng các mô hình ngôn ngữ lớn để hỗ trợ xác định nguồn và phân chìm, đồng thời sử dụng các mô hình tinh chỉnh để phân tích sự lan truyền của dữ liệu bị hư hỏng qua các chức năng trong chuỗi cuộc gọi nguy hiểm ở cấp độ ngữ nghĩa. các

system further utilizes a graph database for efficient storage and querying, with the retrieved connected paths subsequently evaluated by a dedicated vulnerability inference agent. Experimental results demonstrate that Liva achieves notable advantages in detecting taint-related vulnerabilities in IoT firmware. It not only reduces time cost but also improves detection accuracy. On the benchmark dataset, Liva successfully discovered 64 previously unknown vulnerabilities, of which 39 have been officially assigned CVE/CNVD identifiers. Compared with baseline tools, Liva identified more vulnerabilities in less time while significantly reducing false positives. Moreover, the implementation of Liva highlights the feasibility and practical value of applying fine-tuned small-parameter large models to taint analysis tasks.

REFERENCES

- [1] L. S. Vailshery, "Internet of Things (IoT) and non-IoT active device connections worldwide from 2010 to 2025," 2025. [Online]. Available: <https://www.statista.com/statistics/1101442/iot-number-of-connected-devices-worldwide/>
- [2] C. P. S. T. Ltd., Check point software's 2025 security report finds alarming 44% increase in cyber-attacks, 2025. [Online]. Available: <https://www.checkpoint.com/press-releases/check-point-software-2025-security-report-finds-alarming-44-increase-in-cyber-attacks-amid-maturing-cyber-threat-ecosystem/>
- [3] C. P. Research, 6 cyber security challenges emerge from world economic forum: 2025 threats, 2025. [Online]. Available: <https://blog.checkpoint.com/security/6-cyber-security-challenges-emerge-from-world-economic-forum-check-point-research-on-2025-threats/>
- [4] X. T. Feng, X. G. Zhu, Q.-L. Han, W. Zhou, S. Wen, and Y. Xiang, "Detecting vulnerability on IoT device firmware: A survey," *IEEE/CAA J. Automatica Sinica*, vol. 10, no. 1, pp. 25–41, Jan. 2023.
- [5] T. Bakhshi, B. Ghita, and I. Kuzminykh, "A review of IoT firmware vulnerabilities and auditing techniques," *Sensors*, vol. 24, no. 2, Jan. 2024, Art. no. 708.
- [6] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, "FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis," in *Proc. 36th Annu. Comput. Secur. Appl. Conf.*, 2020, pp. 733–745.
- [7] T. Yavuz and C. Brant, "Security analysis of IoT frameworks using static taint analysis," in *Proc. 12th ACM Conf. Data Appl. Secur. Privacy*, Baltimore MD USA, 2022, pp. 203–213.
- [8] K. Cheng et al., "DTaint: Detecting the taint-style vulnerability in embedded device firmware," in *Proc. 48th Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw.*, Luxembourg City, 2018, pp. 430–441.
- [9] X. Wang, J. Zhai, and H. Yang, "Detecting command injection attacks in web applications based on novel deep learning methods," *Sci. Rep.*, vol. 14, no. 1, Oct. 2024, Art. no. 25487.
- [10] S. Chen, Z. Lin, and Y. Zhang, "SelectiveTaint: Efficient data flow tracking with static binary rewriting," in *Proc. 30th USENIX Secur. Symp.*, USENIX Association, 2021, pp. 1665–1682.
- [11] P. Liu et al., "Fits: Inferring intermediate taint sources for effective vulnerability analysis of IoT device firmware," in *Proc. 28th ACM Int. Conf. Architectural Support Programm. Lang. Operating Syst.*, New York, NY, USA, 2024, pp. 138–152.
- [12] N. Redini et al., "Karonte: Detecting insecure multi-binary interactions in embedded firmware," in *Proc. IEEE Symp. Secur. Privacy*, San Francisco, CA, USA, 2020, pp. 1544–1561.
- [13] L. Chen et al., "Sharing more and checking less: Leveraging common input keywords to detect bugs in embedded systems," in *Proc. 30th USENIX Secur. Symp.*, USENIX Association, 2021, pp. 303–319.
- [14] K. Cheng et al., "Detecting vulnerabilities in linux-based embedded firmware with SSE-Based on-demand alias analysis," in *Proc. 32nd ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, Seattle WA USA, 2023, pp. 360–372.
- [15] A. Qasem, M. Debbabi, and A. Soeanu, "Octopustaint: Advanced data flow analysis for detecting taint-based vulnerabilities in IoT/IoT firmware," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2024, pp. 2355–2369.
- [16] J. Ye, X. Fei, X. d. C. de Carnavalet, L. Zhao, L. Wu, and M. Zhang, "Detecting command injection vulnerabilities in linux-based embedded firmware with LLM-based taint analysis of library functions," *Comput. Secur.*, vol. 144, Sep. 2024, Art. no. 103971.
- [17] T. Bakhshi, B. Ghita, and I. Kuzminykh, "A review of IoT firmware vulnerabilities and auditing techniques," *Sensors*, vol. 24, no. 2, 2024, Art. no. 708.
- [18] Y. Gui, D. He, and J. Xue, "Merge-replay: Efficient IFDS-based taint analysis by consolidating equivalent value flows," in *Proc. 38th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2024, pp. 319–331.
- [19] K. Hough and J. Bell, "A practical approach for dynamic taint tracking with control-flow relationships," *ACM Trans. Softw. Eng. Methodol.*, vol. 31, no. 2, Dec. 2021, Art. no. 26.
- [20] X. Li, Q. Wei, Z. Wu, and W. Guo, "Finding taint-style vulnerabilities in lua application of IoT firmware with progressive static analysis," *Appl. Sci.*, vol. 13, no. 17, 2023, Art. no. 9710.
- [21] Hex-Rays SA, "IDA pro: Interactive disassembler and debugger," 2024. [Online]. Available: <https://hex-rays.com/ida-pro/>
- [22] NSA Directorate Research, "Ghidra: A software reverse engineering framework," *Open Source Softw.*, 2019. [Online]. Available: <https://www.nsa.gov/resources/everyone/ghidra/>
- [23] H. Xiao et al., "Accurate and efficient recurring vulnerability detection for IoT firmware," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2024, pp. 3317–3331.
- [24] W. Gibbs et al., "Operation mangos: Scalable discovery of taint-style vulnerabilities in binary firmware services," in *Proc. 33rd USENIX Secur. Symp.*, Philadelphia, PA, USENIX Association, 2024, pp. 7123–7139.
- [25] J. Zhao et al., "Leveraging semantic relations in code and data to enhance taint analysis of embedded systems," in *Proc. 33rd USENIX Secur. Symp.*, Philadelphia, PA, USENIX Association, Aug. 2024, pp. 7067–7084.
- [26] P. Liu et al., "LLM-powered static binary taint analysis," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 3, Feb. 2025, Art. no. 83.
- [27] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, Curran Associates, Inc., 2020, pp. 1877–1901.
- [28] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 1, Jan. 2023, Art. no. 240.
- [29] X. Du et al., "Evaluating large language models in class-level code generation," in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, New York, NY, USA, 2024, pp. 982–994.
- [30] Q. Feng, R. Zhou, C. Xu, Y. Cheng, B. Testa, and H. Yin, "Scalable graph-based bug search for firmware images," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, 2016, pp. 480–491.
- [31] P. Ferrara, A. K. Mandal, A. Cortesi, and F. Spoto, "Static analysis for discovering IoT vulnerabilities," *Int. J. Softw. Tools Technol. Trans.*, vol. 23, no. 1, pp. 71–88, Feb. 2021.
- [32] I. Gotovchits, R. Van Tonder, and D. Brumley, "Saluki: Finding taint-style vulnerabilities with static property checking," in *Proc. Workshop Binary Anal. Res.*, Internet Society, San Diego, CA, 2018, pp. 1–14.
- [33] G. D. Vito, F. Palomba, and F. Ferrucci, "The role of large language models in addressing IoT challenges: A systematic literature review," *Future Gener. Comput. Syst.*, vol. 171, 2025, Art. no. 107829.
- [34] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–31, May 2025.
- [35] A. Shestov, R. Levichev, R. Mussabayev, E. Maslov, A. Cheshkov, and P. Zadorozhny, "Finetuning large language models for vulnerability detection," 2024, *arXiv:2401.17010*. [Online]. Available: <https://arxiv.org/abs/2401.17010>
- [36] Y. Zheng et al., "LlamaFactory: Unified efficient fine-tuning of 100 language models," Jun. 2024, *arXiv:2403.13372*.
- [37] Neo4j Inc., "Neo4j graph database platform," *Graph Database Management System, Version 5.x*, 2024. [Online]. Available: <https://neo4j.com>
- [38] DeepSeek-AI, "Deepseek-R1-distill-qwen-32B," 2024, Accessed: Aug. 20, 2025. [Online]. Available: <https://huggingface.co/deepseek-ai/DeepSeek-R1-Distill-Qwen-32B>
- [39] DeepSeek-AI, "deepseek-coder-33b-instruct," 2024, Accessed: Aug. 20, 2025. [Online]. Available: <https://huggingface.co/deepseek-ai/deepseek-coder-33b-instruct>
- [40] D. Guo et al., "Deepseek-coder: When the large language model meets programming—the rise of code intelligence," 2024, *arXiv:2401.14196*. [Online]. Available: <https://arxiv.org/abs/2401.14196>
- [41] Qwen Team, "QwQ-32B," 2025, Accessed: Aug. 20, 2025. [Online]. Available: <https://huggingface.co/Qwen/QwQ-32B>
- [42] Q. Team, "Qwen3 technical report," 2025, *arXiv:2505.09388*. [Online]. Available: <https://arxiv.org/abs/2505.09388>

- [43] OpenAI, “GPT-4o: Omni-modal AI model,” 2024. [Online]. Available: <https://openai.com/index/hello-gpt-4o/>
- [44] Google DeepMind, “Gemini 2.5 pro: Our most intelligent AI model,” 2025, [Online]. Available: <https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/>
- [45] Qwen Team, “Qwen 2.5-max: Exploring the intelligence of large-scale moe,” 2025, Accessed: Aug. 20, 2025. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5-max/>
- [46] Anthropic, “Claude 3.7 Sonnet system card,” *System card (PDF) by Anthropic*, 2025. [Online]. <https://www.anthropic.com/claude-3-7-sonnet-system-card>
- [47] S. Arzt et al., “FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps,” in *Proc. 35th ACM SIGPLAN Conf. Program. Lang. Des. Implementation*, Edinburgh U.K., 2014, pp. 259–269.
- [48] S. H. Jensen, A. Möller, and P. Thiemann, “Type analysis for javascript,” in *Proc. 16th Int. Symp. Static Anal.*, Berlin, Heidelberg, Springer-Verlag, 2009, pp. 238–255.
- [49] J. Dahse, “RIPS - A static source code analyser for vulnerabilities in PHP scripts,” in *Proc. Seminar Work (Seminer Çalışması)*, Horst Görtz Institute Ruhr-Univ. Bochum, Bochum, Germany, 2010.
- [50] N. Jovanovic, C. Kruegel, and E. Kirda, “Pixy: A static analysis tool for detecting web application vulnerabilities,” in *Proc. IEEE Symp. Secur. Privacy*, Berkeley/Oakland, CA, 2006, pp. 258–263.
- [51] W. Enck et al., “TaintDroid: An information-flow tracking system for realtime privacy monitoring on smartphones,” in *Proc. 9th USENIX Symp. Operating Syst. Des. Implementation*, Vancouver, BC, USENIX Association, 2010, pp. 393–407.
- [52] L. Li et al., “IccTA: Detecting inter-component privacy leaks in Android apps,” in *Proc. IEEE/ACM 37th IEEE Int. Conf. Softw. Eng.*, 2015, pp. 280–291.
- [53] Y. Shoshitaishvili et al., “SoL: (State of) the art of war: Offensive techniques in binary analysis,” in *Proc. IEEE Symp. Secur. Privacy*, 2016, pp. 138–157.
- [54] B. Yadegari, B. Johannesmeyer, B. Whitely, and S. Debray, “A generic approach to automatic deobfuscation of executable code,” in *Proc. IEEE Symp. Secur. Privacy*, 2015, pp. 674–691.
- [55] Z. Gao et al., “Faster and better: Detecting vulnerabilities in linux-based IoT firmware with optimized reaching definition analysis,” in *Proc. Netw. Distrib. Syst. Secur. Symp.*, vol. 26, 2024, pp. 1–16.
- [56] C. Wang, W. Zhang, Z. Su, X. Xu, X. Xie, and X. Zhang, “LLMDFA: Analyzing dataflow in code with large language models,” in *Proc. 38th Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA, Curran Associates Inc., 2025.
- [57] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities,” in *Proc. 13th Int. Conf. Learn. Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=9LdJDU7E91>
- [58] B. Liu et al., “MFTCoder: Boosting code LLMs with multitask fine-tuning,” in *Proc. 30th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, New York, NY, USA, 2024, pp. 5430–5441.
- [59] X. Du et al., “Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG,” 2025, *arXiv:2406.11147*. [Online]. Available: <https://arxiv.org/abs/2406.11147>
- [60] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA, Curran Associates Inc., 2020, pp. 9459–9474.

Zhe Yang received the BS degree from Information Engineering University. He is currently working toward the MS degree with the School of Cyber Science and Technology, Beihang University. His research interests include IoT device security analysis and large language model system applications.

Hao Peng (Senior Member, IEEE) is currently a professor with the School of Cyber Science and Technology, Beihang University. His research interests include network content security and large language model system applications. He is the associate editor of *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, *Neural Networks*, and *International Journal of Machine Learning and Cybernetics*.

Yanling Jiang is currently a research professor with the School of Cyber Science and Technology, Beihang University. Her current research interests include cybersecurity and cyber Governance.

Jianwei Liu (Senior Member, IEEE) received the BS and MS degrees in electronic engineering from Shandong University, China, in 1985 and 1988, respectively, and the PhD degree in communication engineering from Xidian University, China, in 1998. He is currently a professor with the School of Cyber Science and Technology, Beihang University. His current research interests include cryptographic protocol design, security on wireless and mobile network, computer network security and Cryptography.

Hongbin Luo received the BS degree from Beihang University, in 1999 and the MS (with honors) and PhD degrees in communications and information science from the University of Electronic Science and Technology of China (UESTC), in June 2004 and March 2007, respectively. He is currently a professor and dean with the School of Cyber Science and Technology, Beihang University. His research interests are in the wide areas of network technologies including cybersecurity, routing and Internet architecture.

Mingsheng Tang (Member, IEEE) received the BS degree in computer science and technology from Xi'an Jiaotong University, in 2008, and the MS and PhD degrees in computer science and technology from the National University of Defense Technology, in 2010 and 2015, respectively. He currently is an associate professor with the School of Cyber Science and Technology, Beihang University. His current research interests include cybersecurity and multi-agents system.

Jiahe Li received the BS degree from Information Engineering University. He is currently working toward the MS degree with the School of Cyber Science and Technology, Beihang University. His research interests include cybersecurity and large language model system applications.

Kun Zhang received the BS degree from the National University of Defense Technology. He is currently working toward the MS degree with Beihang University. His research interests include applications related to multi-agent and large language model systems.